Introdução à plataforma Arduino

v.2.1
julho/2013

Leonardo Henrique Day de Toledo
Robson Paes Machado
Leandro Ramos

A obra Introdução à plataforma Arduino foi licenciada com uma Licença Creative Commons - Atribuição - Uso Não Comercial - Partilha nos Mesmos Termos 3.0 Não Adaptada.
Com base na obra disponível em www.eaduino.com.br.

1 INTRODUÇÃO AO ARDUINO 1

1.1 O QUE É O ARDUINO? 1
1.2 PREPARANDO O AMBIENTE DE PROGRAMAÇÃO ARDUINO 5
1.3 TESTANDO A PLACA ARDUINO 16

2 NOÇÕES DE ELETRICIDADE 22

2.1 CORRENTE, TENSÃO E RESISTÊNCIA 22
2.1.1 Corrente 22
2.1.2 Tensão 22
2.1.3 Corrente Contínua x Corrente Alternada 22
2.1.4 Resistência 26

3 PLACA PARA TESTES (PROTOTIPAGEM) 27

4 ATIVIDADES 29

4.1 ATIVIDADE 1 - ACENDENDO UM LED 30
4.1.1 Resistor 30
4.1.2 LED 32
4.1.3 O resistor como limitador de corrente 34
4.1.4 Estrutura de um sketch 36
4.1.5 setup() 36
4.1.6 loop() 36
4.1.7 Comentários 40
4.1.8 setup() 40
4.1.9 loop() 40
4.1.10 digitalWrite() 41
4.2 ATIVIDADE 2 - PISCA LED 42
4.2.1 Variáveis 43
4.2.2 int 44
4.2.3 delay() 44
4.3 ATIVIDADE 3 - CONTROLE DE LUMINOSIDADE DE LEDS 45
4.3.1 for() 47
4.3.2 Operadores 48
4.3.3 analogWrite() 48
4.4 ATIVIDADE 4 - LIGANDO/DESLIGANDO UM LED ATRAVÉS DE UMA CHAVE 50
4.4.1 Chave Táctil 50
4.4.2 digitalRead() 53
4.4.3 if() 53
4.4.4 Resistores pull-up / pull-down 54
4.4.5 Atividade 5 - Controle de luminosidade de LED com LDR 57
4.4.6 LDR 57
4.4.7 float 60
4.4.8 analogRead() 60
4.5 ATIVIDADE 6 - SIMULANDO UM SEMÁFORO 61
4.6 ATIVIDADE 7 – MEDINDO TEMPERATURAS COM NTC 65
4.6.1 NTC 65
4.6.2 #include 68
4.6.3 double 68
4.6.4 Functions 68
4.6.5 log() 69
4.6.6 Serial 69

5 CONCLUSÃO 70

1 Introdução ao Arduino

1.1 O que é o Arduino?

Arduino é uma plataforma de hardware e software *open source*, programável, que permite interagir com o mundo físico (luzes, motores, etc), tanto com programas residentes no próprio Arduino, quanto utilizando este como interface entre o mundo físico e um computador ou com outras plataformas de desenvolvimento (Raspberry Pi, por exemplo).

Open source quer dizer que você pode copiar, modificar, vender o seu próprio Arduino. A única restrição é o uso do nome Arduino, que é exclusivo da equipe que desenvolveu esta plataforma. Por isso você vai encontrar no mercado, os originais Arduino, produzidos na Itália, e vários compatíveis, porém com outros nomes e países de origem como: Freeduino, Severino, Sanguino, Roboduino, Seeeduino, Garagino, Program-me e por aí vai.

Os originais italianos que mais se destacam são: **Uno, Mega** e o **Leonardo**. No mercado ainda encontramos muitas placas compatível com o Arduino **Duemilanove**, uma placa mais antiga com excelente preço. Para uma lista completa e diferenças entre os originais Arduinos, visite http://arduino.cc/en/Main/Boards

A plataforma Arduino UNO apresenta uma ótima relação custo x benefício, sendo assim um excelente ponto de partida para o *hobbista* iniciante. As placas Duemilaenove e UNO são praticamente iguais. Usam o mesmo microcontrolador da **Atmel (**http://www.atmel.com/) , o **ATmega328p** (versões mais antigas usam o ATmega168), e a mesma disposição dos pinos de entrada e saída. A diferença está no *chip* que faz a comunicação da placa Arduino com o computador. A placa Duemilanove utiliza um chip da **FTDI** (http://www.ftdichip.com/), UNO utiliza o chip **ATmega8U2** da Atmel e Leonardo integrou as funcionalidades dos dois controladores da placa UNO em um único chip, o **ATmega32u4**.

Não se preocupe em memorizar toda esta informação. Conforme você vai utilizando a plataforma estes detalhes irão ficar familiares.

Apenas para ilustrar, a seguir as duas placas mais utilizadas:

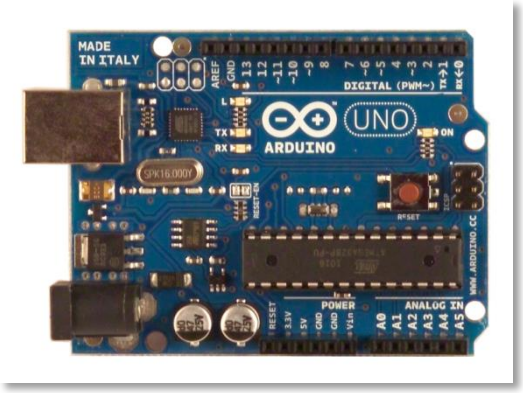

Figura 1-1 – Arduino UNO

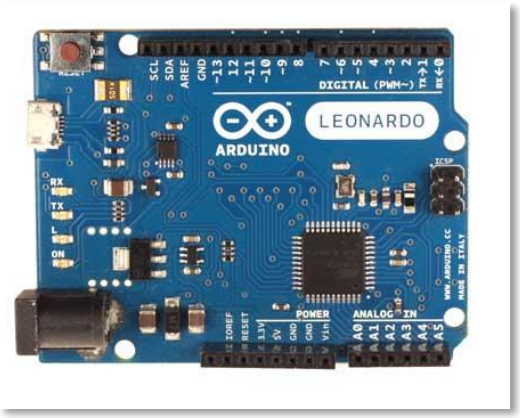

Figura 1-2 – Arduino Leonardo

Os autores do projeto Arduino pensaram numa forma que qualquer pessoa interessada, pudesse fazer uso desta tecnologia sem, necessáriamente, ser alguém que entendesse de eletrônica ou programação de computadores. Outro ponto de destaque é que quando comparado a outras plataformas semelhantes de desenvolvimento, é muito acessível do ponto de vista financeiro. Além disso, seu software de programação é gratuíto e disponível para os sistemas Windows, Linux e MacOS.

Então podemos entender que a plataforma Arduino é composta de uma placa original ou compatível (hardware) e um ambiente de desenvolvimento (software).

O fantástico desta plataforma é que a comunidade de usuários/desenvolvedores Arduino, constantemente pensam em uma forma simples de ampliar/extender suas funcionalidades, acoplando outros elementos eletrônicos em placas conhecidas como s*hields*. Os *shields* são literalmente empilhados no Arduino, criando novas possibilidades de uso. Existem *shields* que fazem controle de motores, luzes, tomadas, acesso a internet com e sem fio, etc.

Figura 1-3 – Shields

Não deixe de visitar http://www.shieldlist.org/. Este site contém uma lista atualizada de *shields* desenvolvidos.

A linguagem de programação do Arduino é muito simples. Possui poucos comandos, facilitando em muito o aprendizado. Programadores experientes na linguagem C, podem extender as bibliotecas que já fazem parte da linguagem ou até mesmo criar novas ou modificar as existentes. As bibliotecas contém códigos que tornam a escrita dos programas mais simples, fazendo que os detalhes de comunicação com o hardware sejam facilitados, deixando de lado o trabalho "sujo" e "pesado".

O ambiente de desenvolvimento do Arduino é um software que contém um editor de texto para escrever o programa, uma área de mensagens, uma barra com botões para funções mais comuns, e uma série de menus. Este software é responsável pela transformação do código escrito em algo que o microcontrolador possa entender. A transferência do programa para a placa se dá através de um cabo USB, conectando o seu computador com a placa Arduino.
Se você já usa o Microsoft Office ou OpenOffice ou qualquer outra aplicação para Microsoft Windows, por exemplo, não terá dificuldades em operar este ambiente. Aliás, programas ou scripts são chamados de *sketches* (esboços) no universo Arduino.

1.2 Preparando o ambiente de programação Arduino

O primeiro passo é fazer o download do ambiente de desenvolvimento. Entre neste endereço http://arduino.cc/en/Main/Software e selecione o pacote para o seu ambiente operacional (Windows, Mac ou Linux).

Supondo que o seu ambiente seja Microsoft Windows, o arquivo baixado poderá ser **arduino-1.0.5-windows.exe** ou **arduino-1.0.5-windows.zip**. A versão .exe é a mais indicada para os que não estão habituados a instalar programas neste ambiente.

Obs. Arduino 1.0.5 é a última versão desta série. As próximas versões serão a partir da 1.5, que inclui a placa Arduino Due.

Após o *download*, execute o **arduino-1.0.5-windows.exe**. Clique no botão **I Agree**

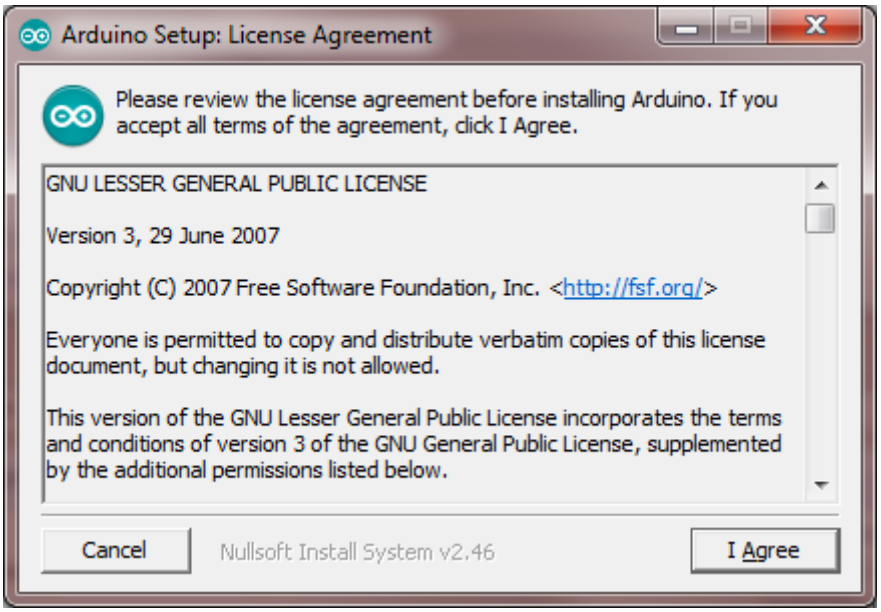

Figura 1-4 – instalando o software Arduino

Mantenha os itens selecionados e clique no botão **Next**

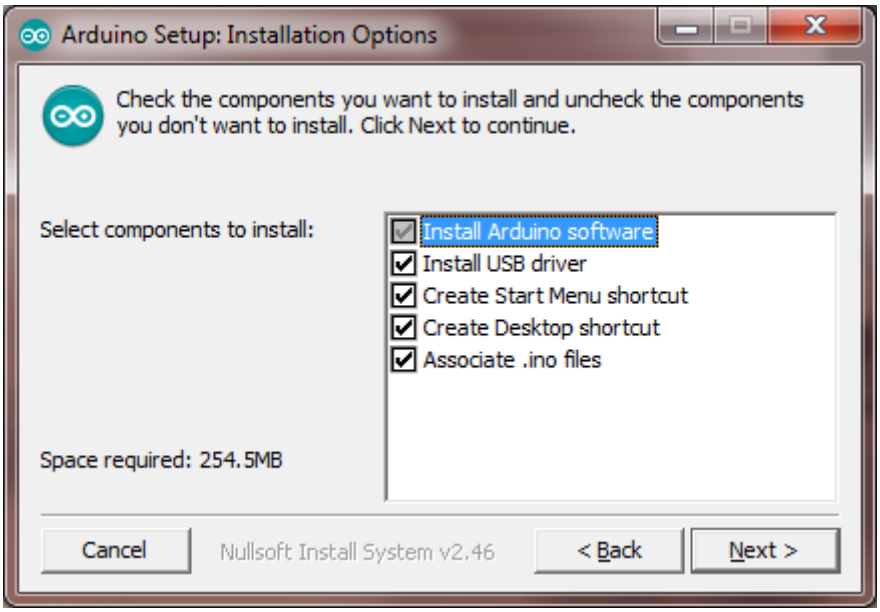

Figura 1-5 – instalando o software Arduino

Escolha uma nova pasta (botão **Browse**) para instalação ou aceite a sugestão.

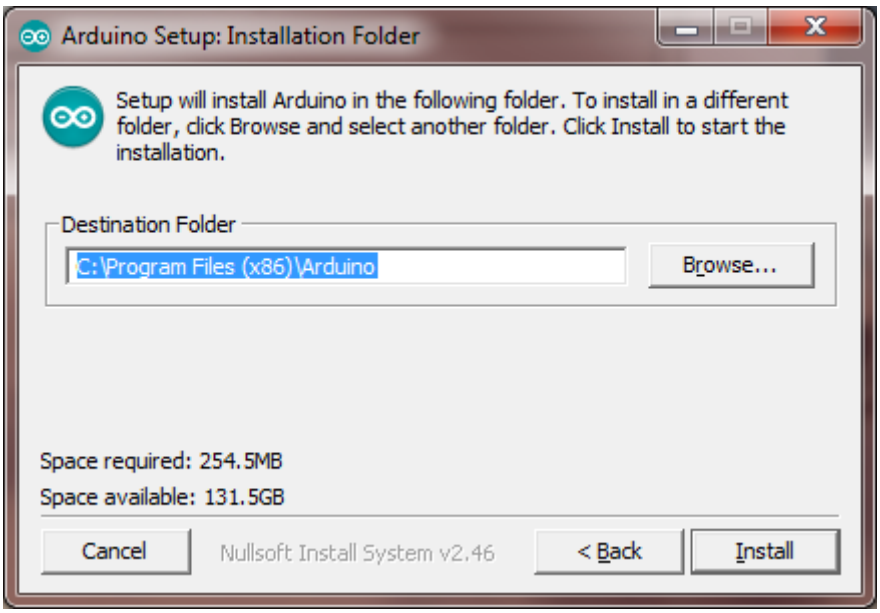

Figura 1-6 – instalando o software Arduino

No nosso exemplo vamos indicar a unidade F, pasta Arduino. Clique em seguida no botão **Install**

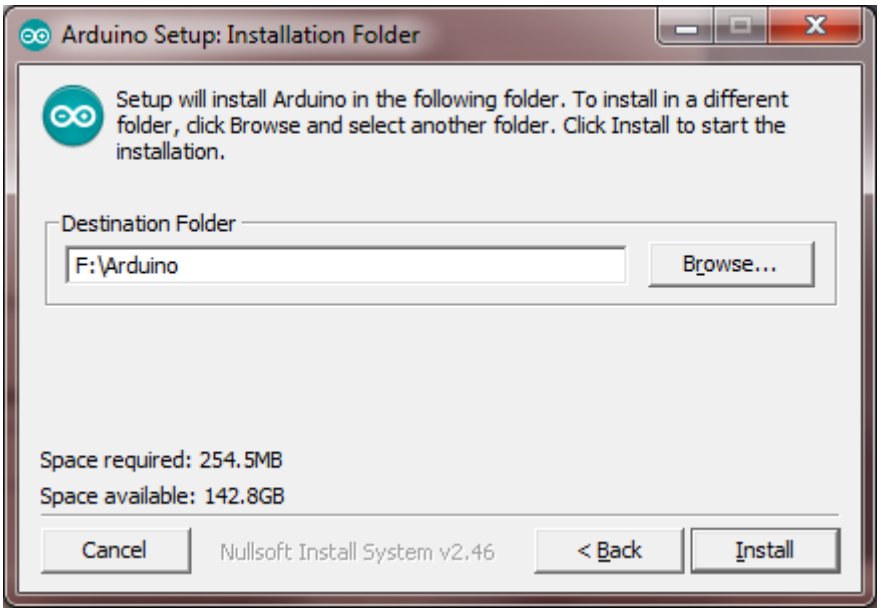

Figura 1-7 – instalando o software Arduino

Aguarde a instalação...

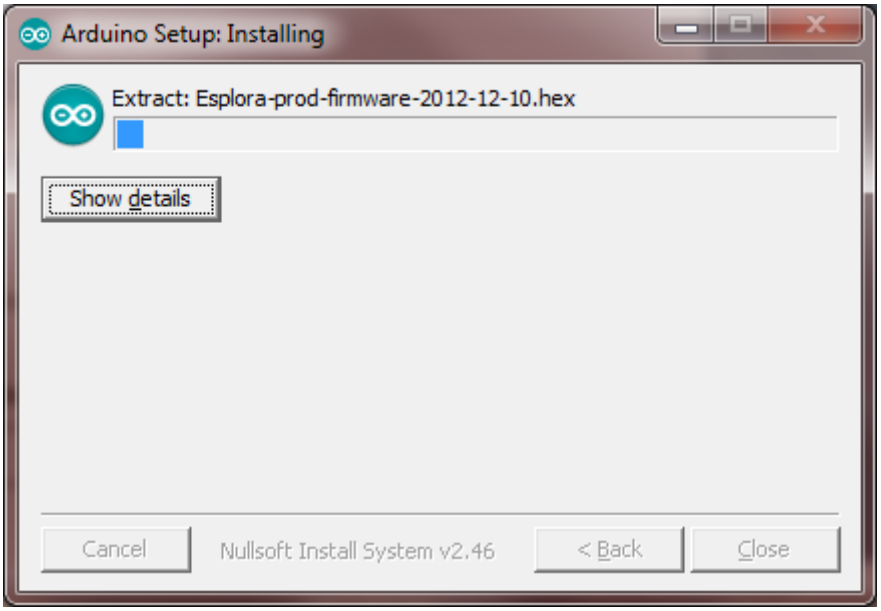

Figura 1-8 – instalando o software Arduino

Caso o Windows apresente a seguinte mensagem...

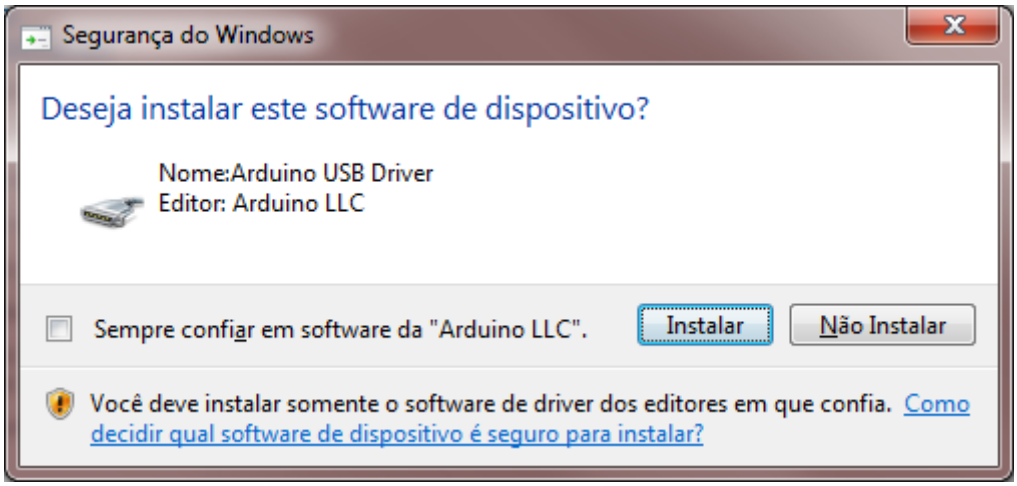

Figura 1-9 – instalando o software Arduino

...clique no botão **Instalar**

Ao final da instalação clique no botão **Close**.

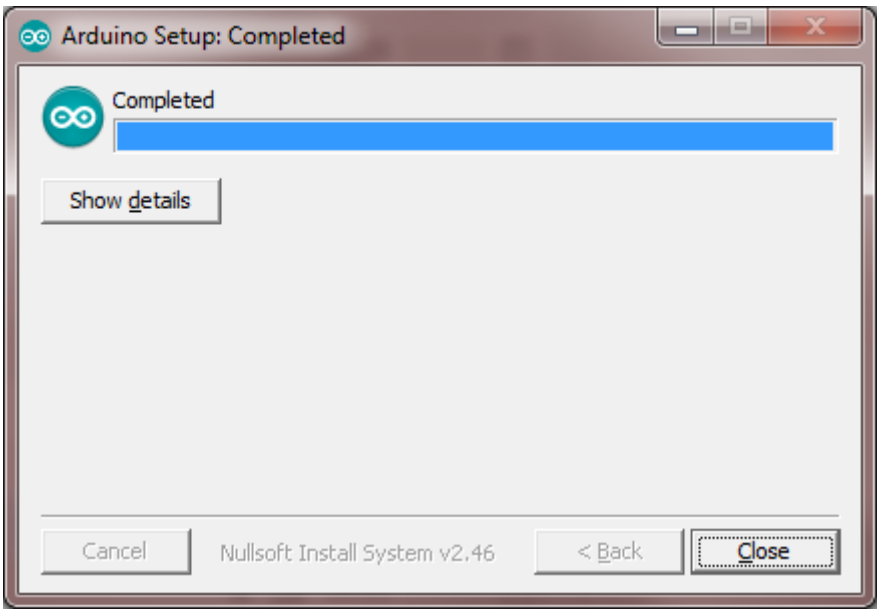

Figura 1-10 – instalando o software Arduino

Pronto! O software Arduino está instalado.

Todos os arquivos necessários para executar o Arduino foram instalados na pasta selecionada. É só dar um duplo clique no arquivo **arduino.exe** para iniciar o software.

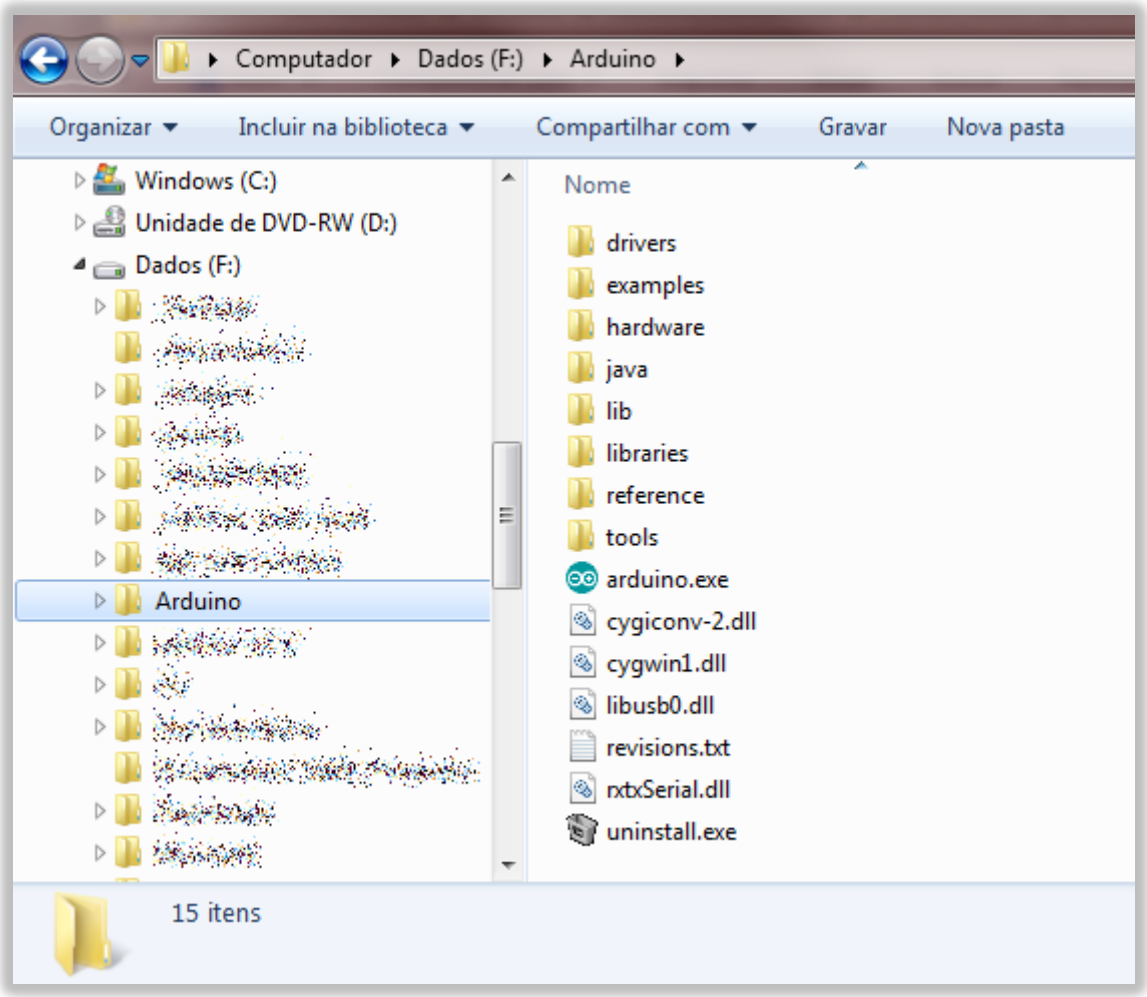

Figura 1-11 – pasta com os arquivos da instalação

Agora será necessário configurar ou verificar a porta de comunicação do seu computador e no geral não oferece dificuldades. Esta é a tarefa requer um pouco mais de atenção. O exemplo a seguir assume que estamos com um **Arduino Duemilanove** ou compatível em ambiente operacional **Microsoft Windows 7**. Este endereço http://arduino.cc/en/Guide/windows detalha a instalação do driver de comunicação para diversos modelos de placa e versões do Microsoft Windows.

Vamos conectar o cabo USB no Arduino e na porta USB do seu computador. Caso seja primeira vez, o Microsoft Windows identificará um novo dispositivo e tentará instalar os drivers correspondentes.

Clique com o botão direito do mouse em **Computador**. Selecione Gerenciar ou Propriedades.

No nosso exemplo vamos selecionar **Gerenciar**.

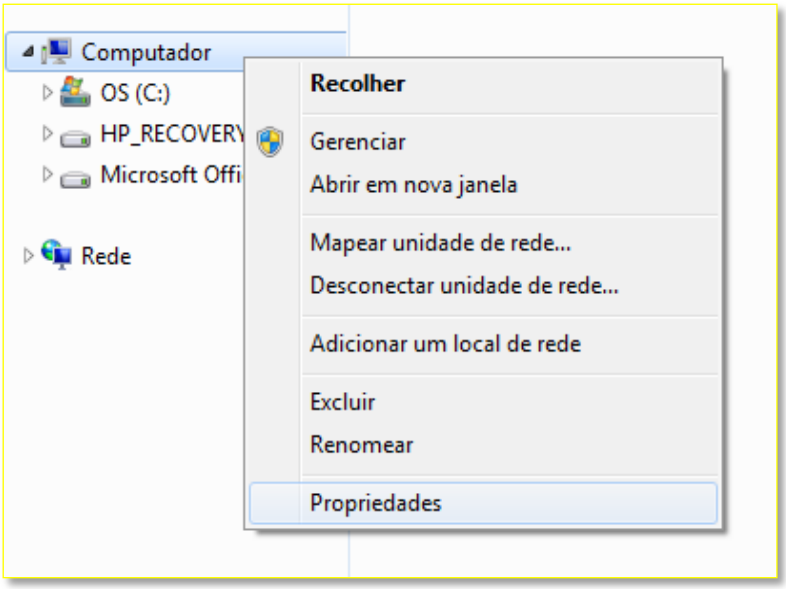

Figura 1-12 – verificando o driver de comunicação

A janela **Gerenciamento do computador** será aberta. Clique em **Portas (COM e LPT)**.

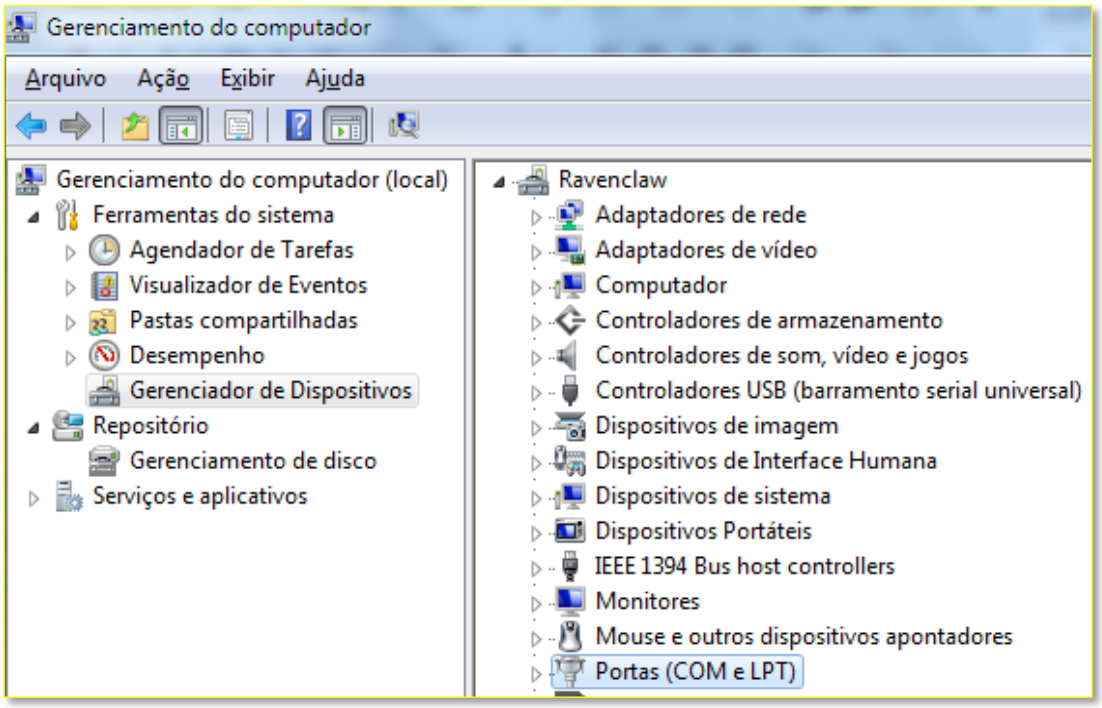

Figura 1-13 – verificando o driver de comunicação

Clique com o botão direito do mouse em **USB Serial Port** e em seguida clique em **Propriedades**.

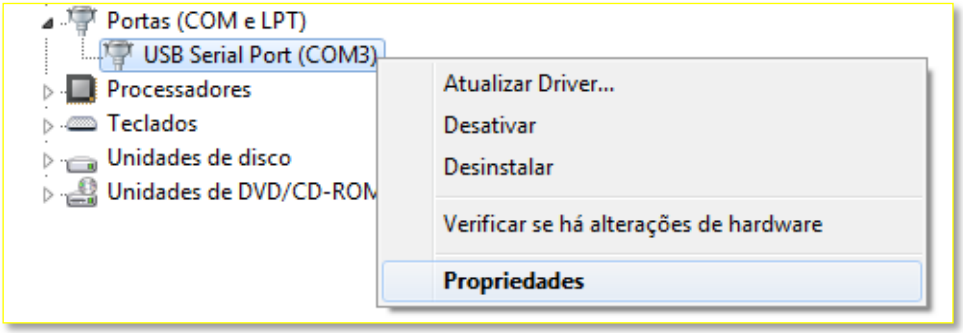

Figura 1-14 – verificando o driver de comunicação

Nesta janela, note o que está escrito em **Status do disposto.**

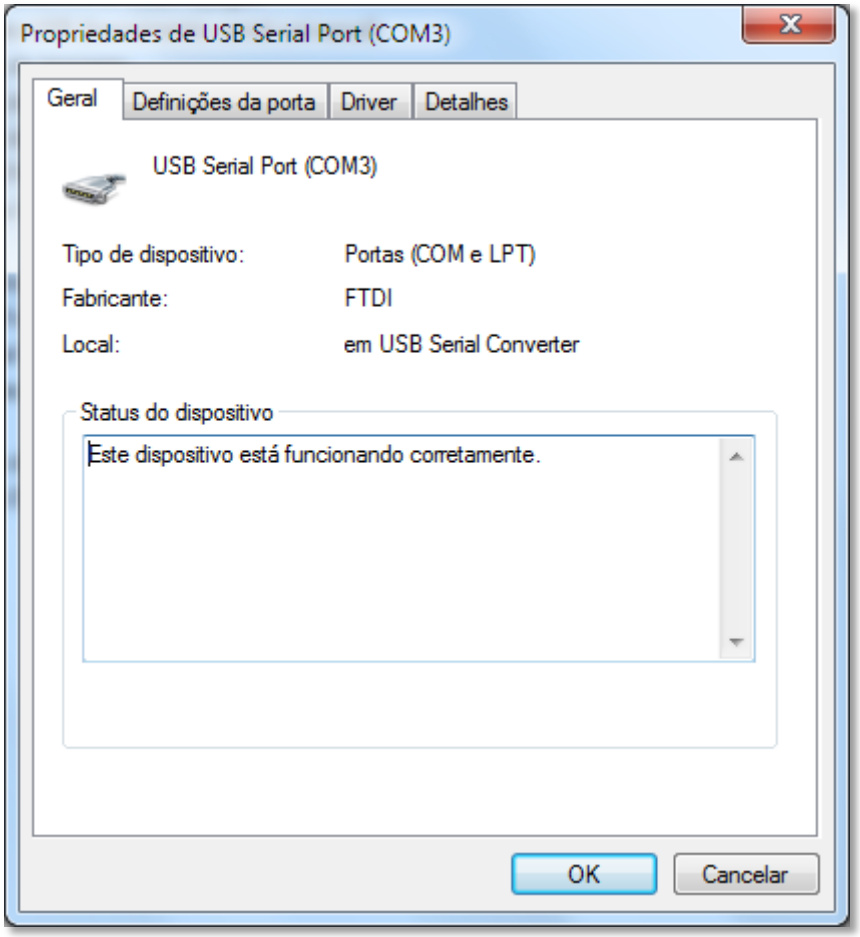

Figura 1-15 – verificando o driver de comunicação

No exemplo, a configuração está correta, mas se não for este o seu caso, clique na aba **Driver**. Note também o campo **Fabricante**: **FTDI**. Dependendo da placa, este campo pode apresentar outro nome.

Clique no botão **Atualizar Driver...**

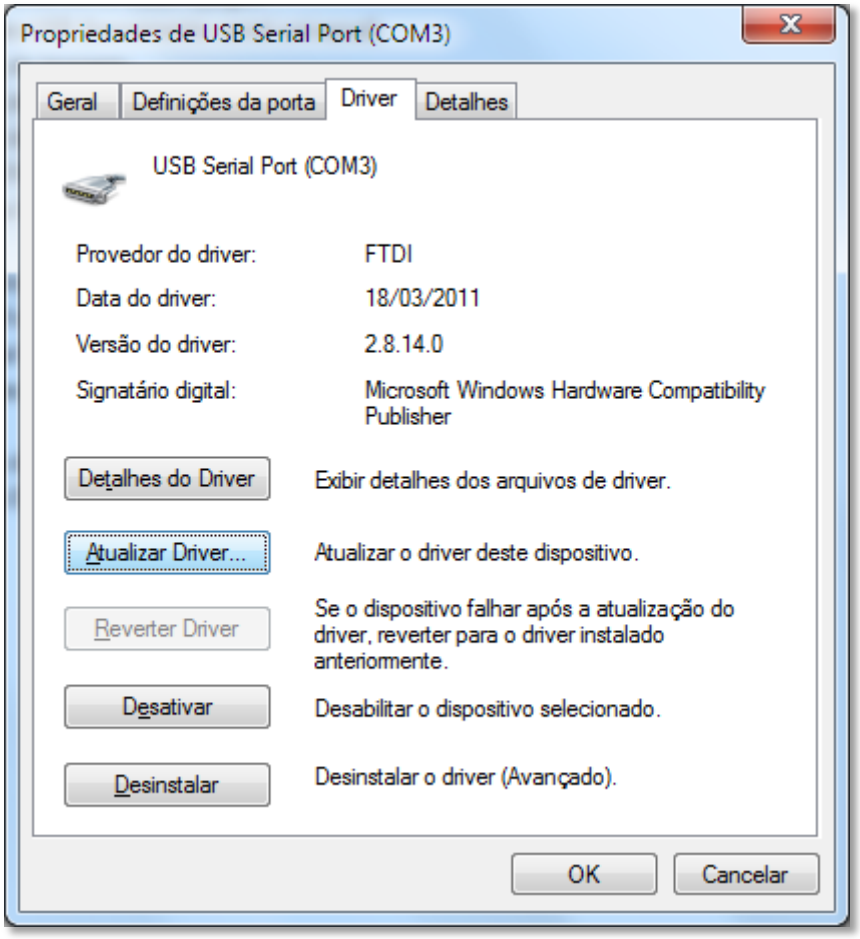

Figura 1-16 – atualizando o driver de comunicação

Clique em **Procurar software de driver no computador**.

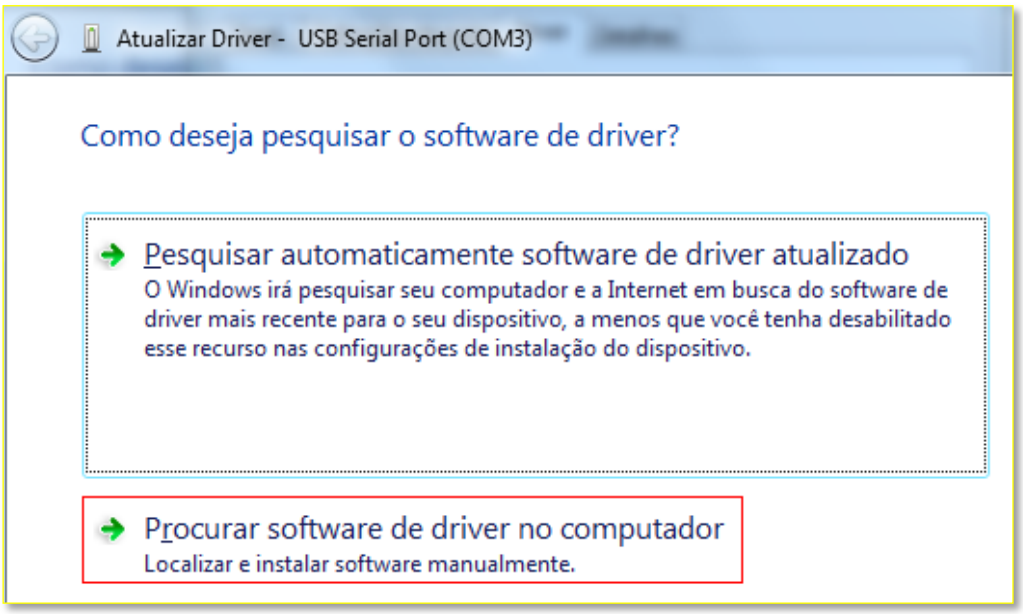

Figura 1-17 – atualizando o driver de comunicação

Nesta janela, clique no botão **Procurar...**

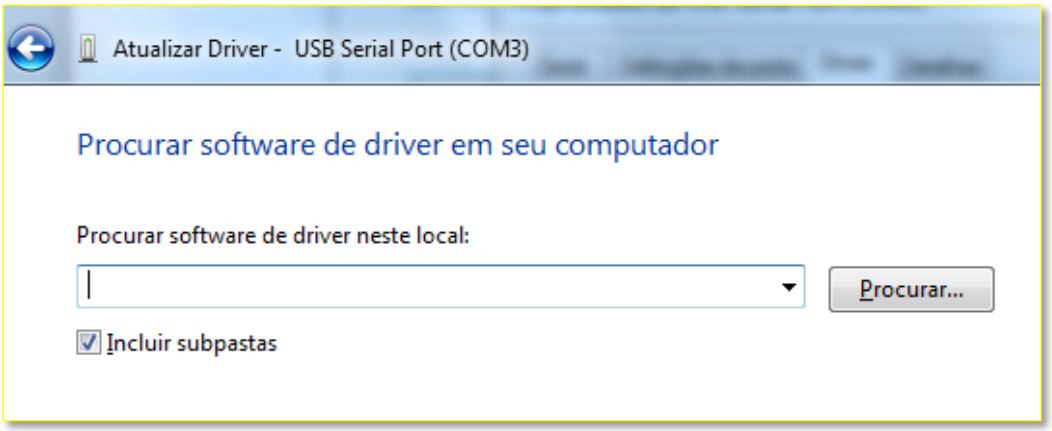

Figura 1-18 – atualizando o driver de comunicação

Navegue até a pasta onde foi instalado o Arduino e selecione a pasta **FTDI USB Drivers** (ou o arquivo .inf adequado a sua placa). Clique no botão **OK**.

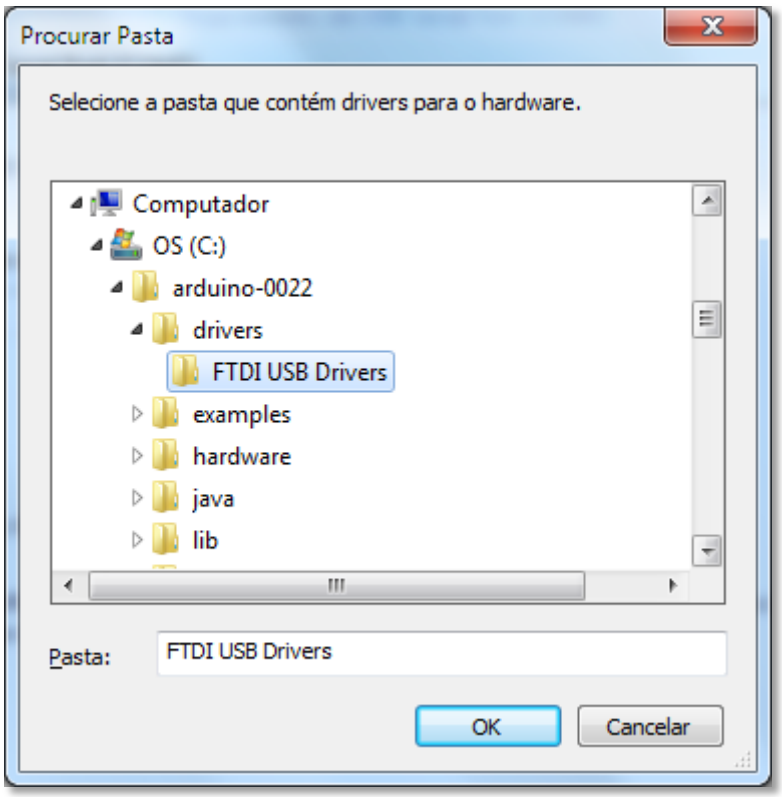

Figura 1-19 – atualizando o driver de comunicação

Se houver alguma dúvida, recorra a este endereço: http://arduino.cc/en/Guide/windows

1.3 Testando a placa Arduino

Vá até a pasta onde foi instalado o Arduino e dê um duplo clique em **arduino.exe** e aguarde e o carregamento do software.

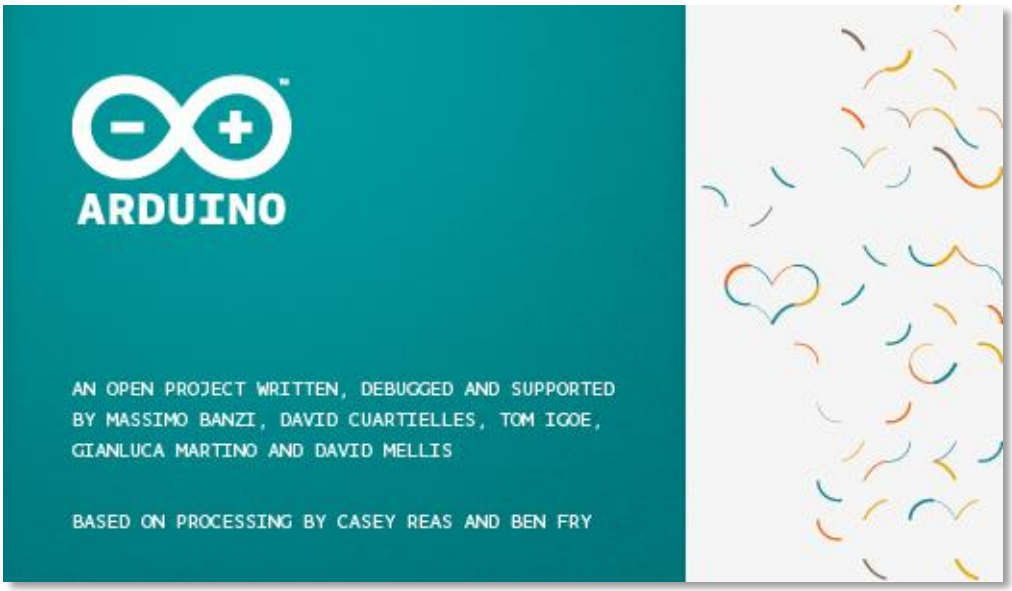

Figura 1-21 – tela de apresentação

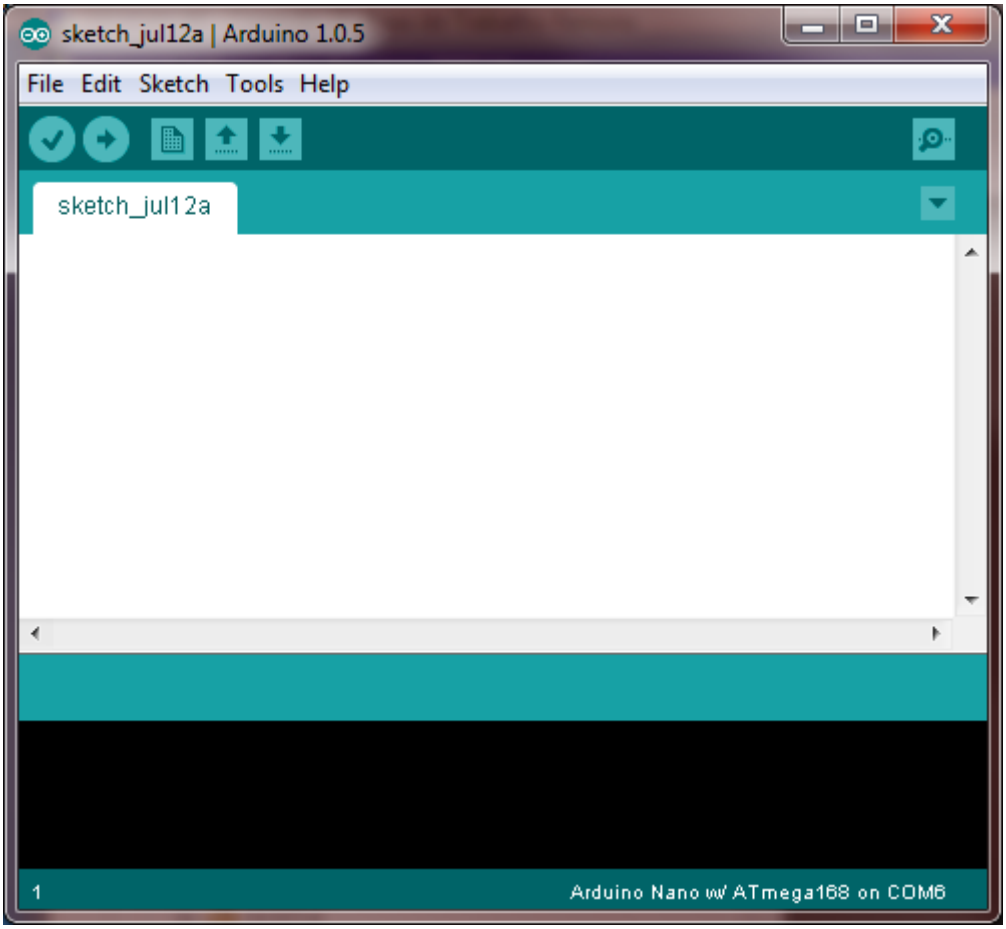

Figura 1-22 – software carregado

Acesse o menu **Tools** e em seguida o item **Board**.

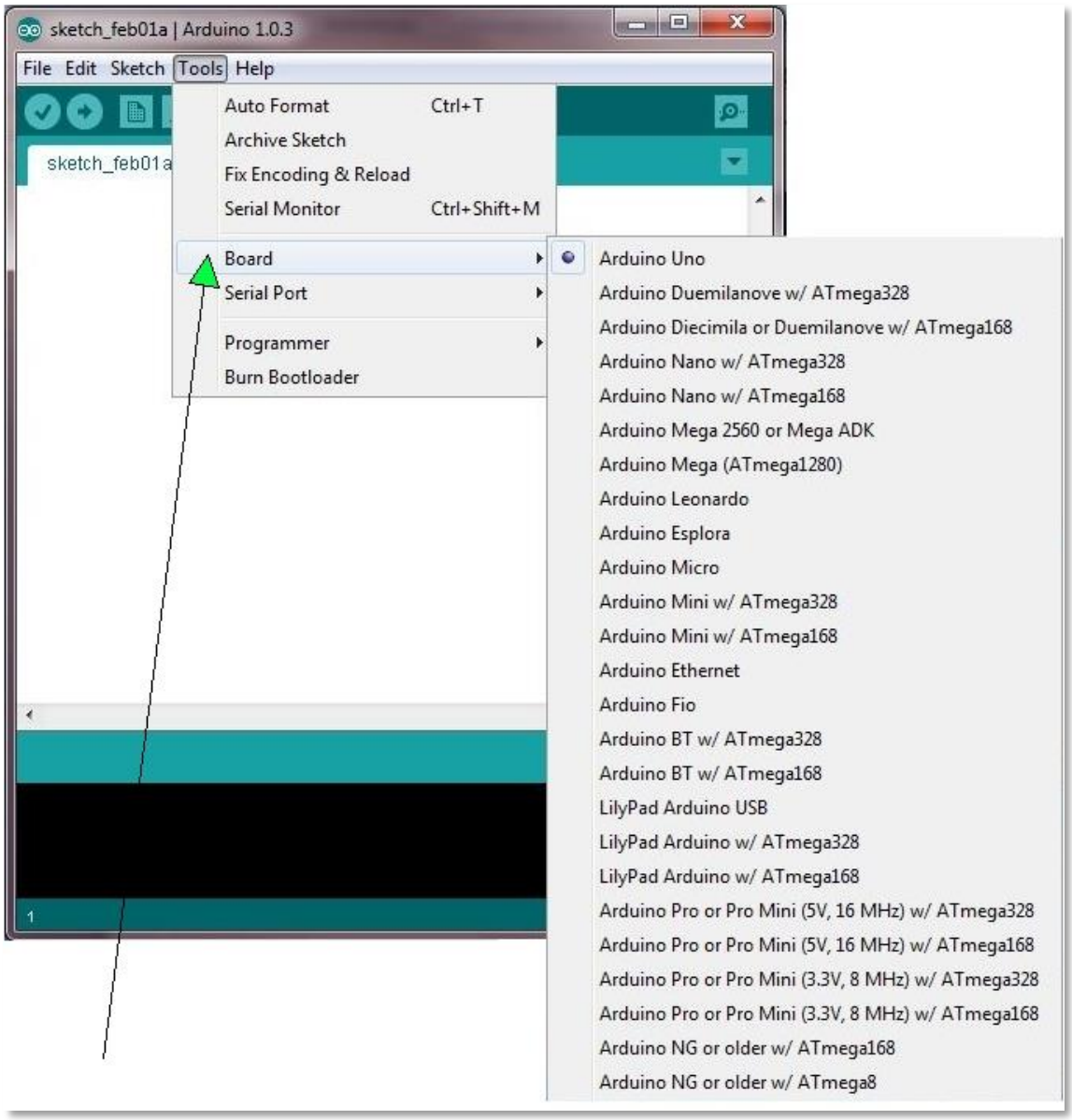

Figura 1-23 – seleção da placa Arduino

O software oferece uma lista de placas Arduinos mais comuns. No nosso exemplo vamos selecionar a **Arduino Duemilanove w/ ATmega328** **(veja qual o modelo da sua placa e selecione a correta)**.

Agora, acesse o menu **Tools** e em seguida o item **Serial Port**.

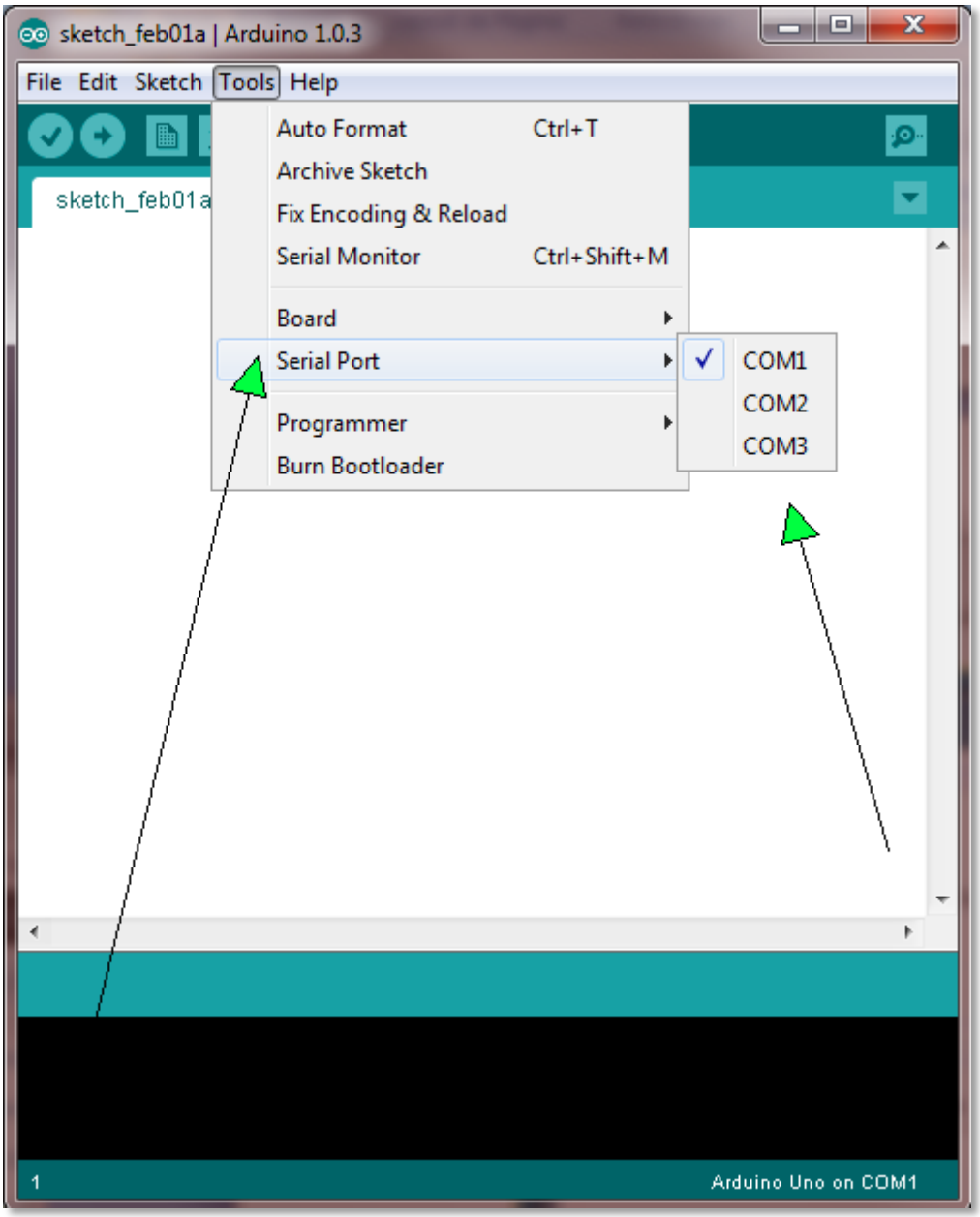

Figura 1-24 – seleção da porta de comunicação

Irá aparecer uma lista de portas COM ativas do seu computador. Caso o próximo passo apresente erros, volte a este item e altere a porta. Porém, se você passou pelo item 1.2 deste tutorial, já saberá qual porta está associada ao Arduino, não é? Se for preciso, volte ao item e reveja os procedimentos.

Agora chegou o momento de verificarmos se a plataforma (hardware e software) está corretamente configurada.

Acesse o menu **File**, selecione **Examples**, **01.Basics** e **Blink**.

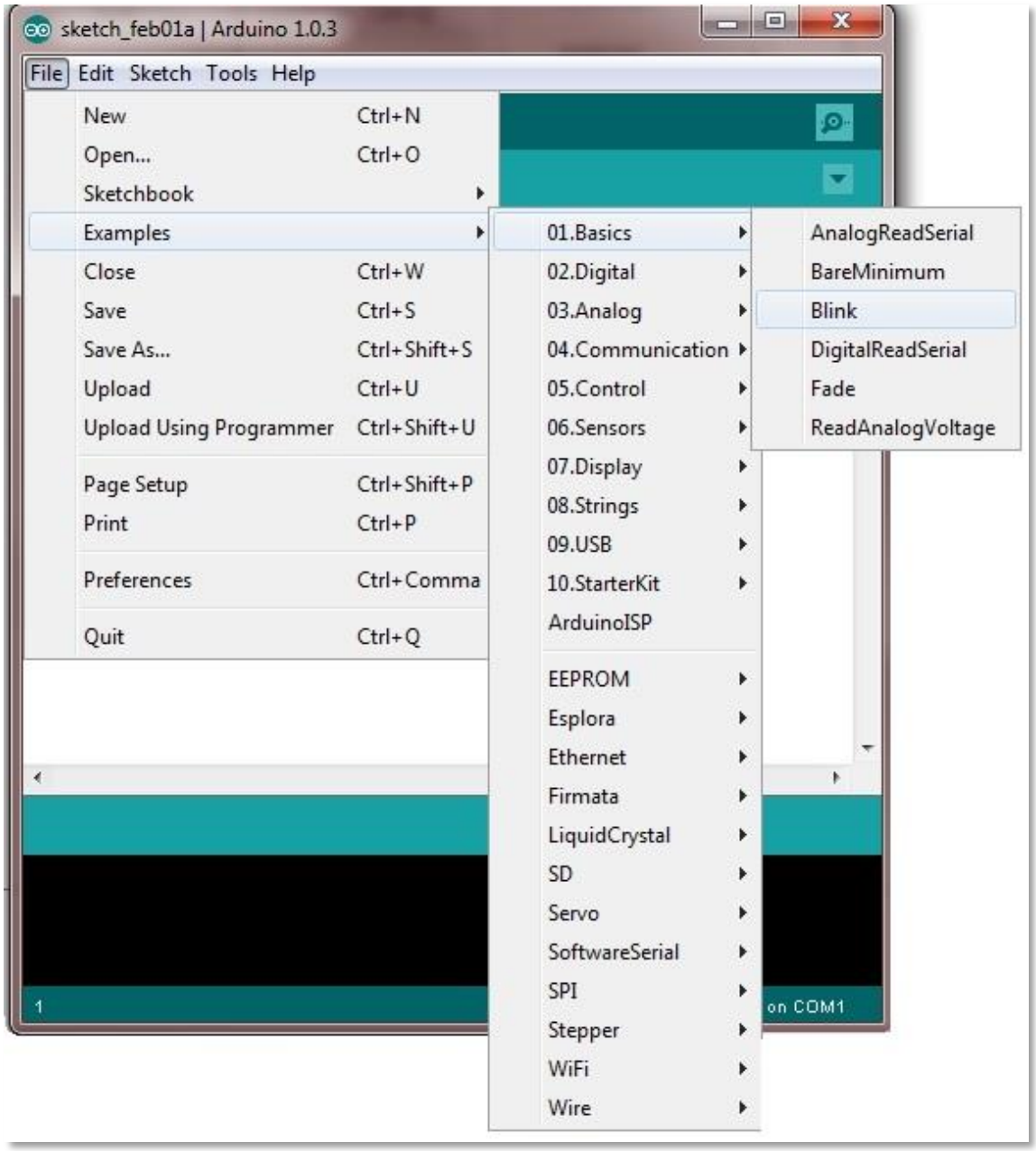

Figura 1-25 – exemplos

Uma nova janela do ambiente será aberta com o *sketch* **Blink**.

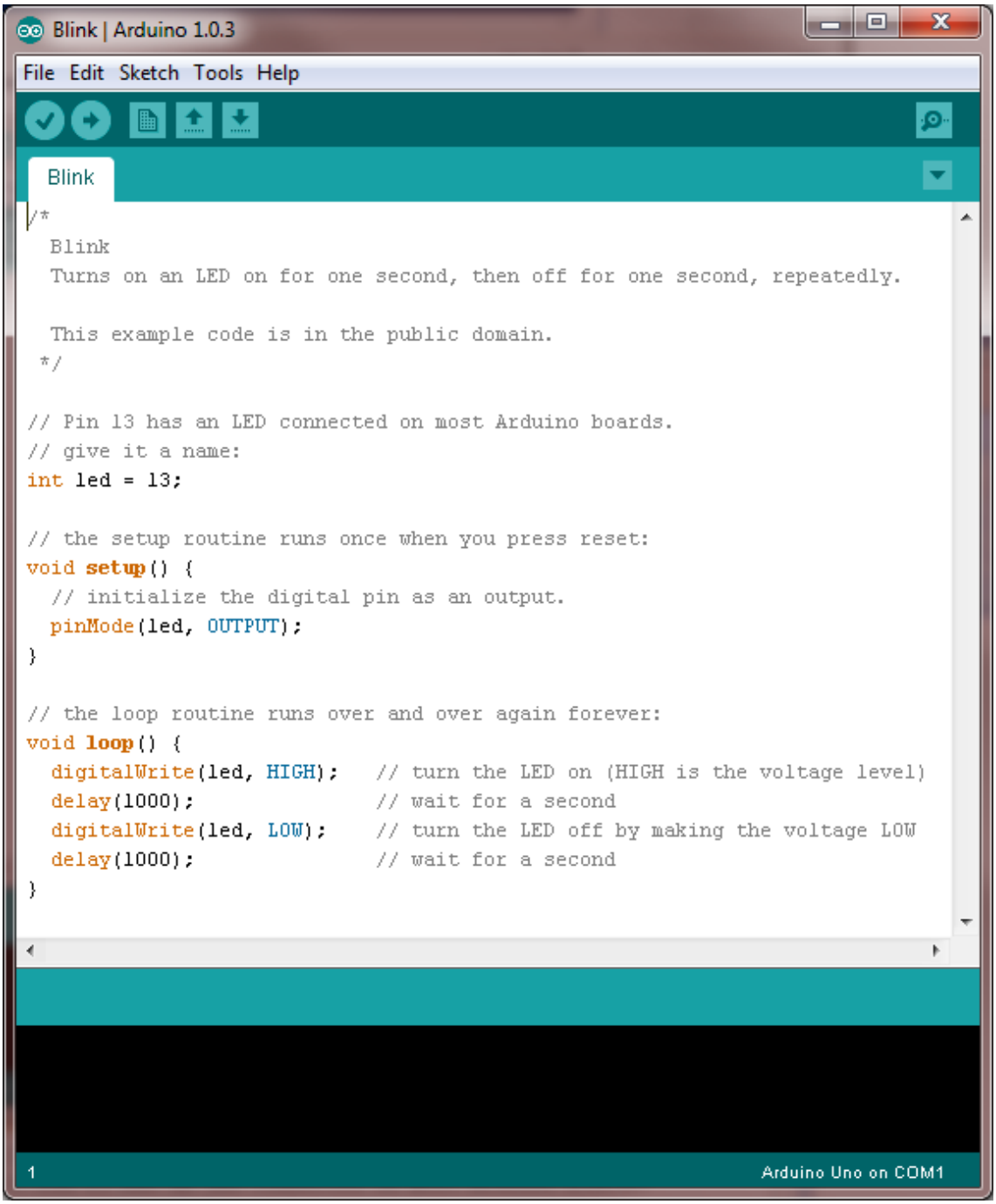

Figura 1-26 – exemplo Blink

Como dito anteriormente, não se preocupe neste momento em entender o que está escrito. Agora vamos transferir o sketch para a placa. Clique no botão em destaque e note o texto ao lado do último botão: **Upload**.

Figura 1-27 – transferindo o *sketch* para a placa

Se tudo estiver corretamente configurado, na parte inferior da tela você verá uma mensagem similar a esta:

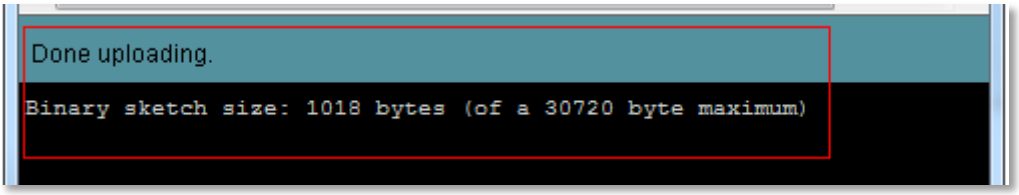

Figura 1-28 – área de mensagens

Isto nos diz que o programa foi transferido, **Done uploading**. Se aparecer valor diferente em **Binary sketch size: 1018 bytes** é certo você estar usando uma placa diferente do nosso exemplo.

Qual o resultado esperado? Você deverá ver um LED piscando na sua placa Arduino. Não é mágica ☺, isto ocorre em todas as placas compatíveis com as originais italianas. Pode acontecer de você estar com uma placa que não possua este componente. Se você não sabe o que é um LED, no próximo módulo explicaremos o que é e como usá-lo.

Caso não tenha tido sucesso com a instalação ou teste da placa, primeiramente reveja todo este módulo e se ainda assim não funcionar envie uma mensagem para suporte@arduino.com.br.

2 Noções de Eletricidade

2.1 Corrente, Tensão e Resistência

Antes de experimentarmos o Arduino, vamos ver alguns conceitos básicos de eletricidade.

2.1.1 Corrente

Para que um dispositivo elétrico/eletrônico funcione, é necessário que uma energia denominada **corrente elétrica**, flua através deste dispositivo.

A corrente elétrica é composta por um fluxo de cargas que circulam entre os **pólos** de um **gerador**, passando pelos componentes que formam o nosso dispositivo. O caminho fechado percorrido pela corrente elétrica é conhecido como **circuito elétrico**.

Os **condutores** são os elementos que permitem a passagem da corrente elétrica.

A unidade de medida é o **ampère (A)** cujo nome foi uma homenagem ao físico francês André-Marie Ampère.

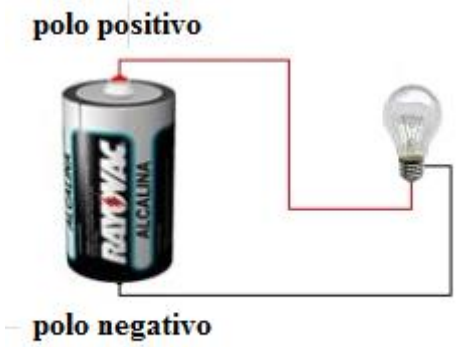

Figura 2-1 – circuito elétrico simples

- pilha = gerador de energia
- fio = condutor
- lâmpada = dispositivo

2.1.2 Tensão

A força que "empurra" as cargas pelo circuito é conhecida como **tensão**. Esta força aparece pela diferença de cargas positivas e negativas que há nos polos do gerador.

A unidade de medida é o **volt (V)**, homenagem ao físico italiano Alessandro Volta.

2.1.3 Corrente Contínua x Corrente Alternada

Tomando como exemplo uma pilha, notamos que este gerador possui polos fixos (positivo e negativo). Sendo assim o fluxo de cargas não muda de direção. Este tipo de corrente é conhecido como **corrente contínua** (**DC** – *direct current*), pois a corrente circula pelo nosso circuito num só sentido.

Em um gerador de **corrente alternada** (**AC** - *alternating current*) os pólos mudam constantemente e consequentemente, a direção da corrente. Esta direção muda 60 vezes a cada segundo. Este número de ciclos (positivo e negativo) por segundo chamamos de **frequência** e a unidade de medida é o **hertz (Hz)**, homenagem ao físico alemão Heinrich Hertz. Temos então que a tensão da rede tem uma frequência de 60Hz (no Brasil).

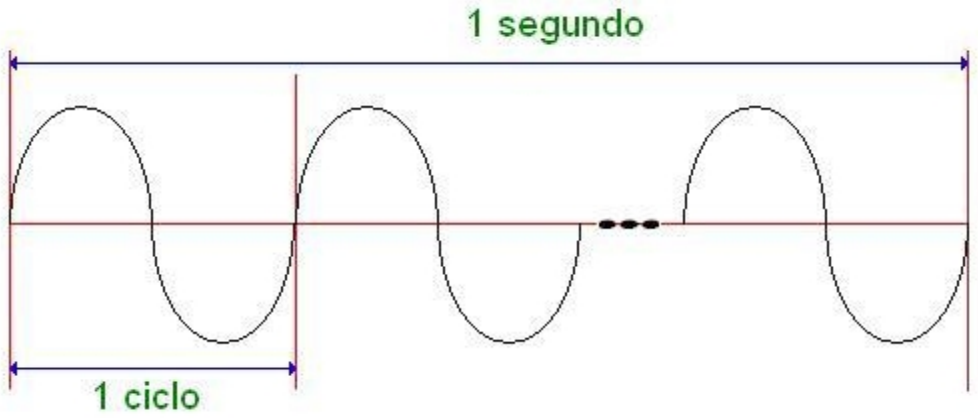

Figura 2-2 – corrente alternada

O tipo de corrente que chega em nossas residências é a alternada com uma tensão de 110V ou 220V, muito elevada e ainda de tipo diferente de que precisamos para o Arduino e os *Shields*. Por isso usamos uma fonte de no máximo 12V ou uma bateria de 9V para alimentar o Arduino.

Se você está atento a este tutorial, e temos certeza que está, notou que no módulo 1 testamos a placa conectado ao computador e não foi preciso uma fonte externa de energia. Isto ocorre porque o cabo USB contém uma linha de energia de 5V que "retira" do seu computador e alimenta a nossa placa. Porém, haverá situações que a corrente exigida por algum componente conectado a placa Arduino, como exemplo um motor, será muito maior que a corrente que a porta USB do seu computador poderá fornecer e aí nestas situações vamos conectar uma fonte externa de energia.

Em todas as atividades propostas neste tutorial, não será necessário o uso de uma fonte externa. O consumo dos componentes eletrônicos é muito baixo e a porta USB do seu computador dará conta do recado.

Como podemos adquirir um Arduino de diversos fornecedores, temos que apenas prestar atenção como a placa está configurada para receber a alimentação: através do cabo USB ou de uma fonte externa. Nas placas originais italianas esta seleção é automática. A placa "sente" se a fonte de energia vem do cabo USB ou de uma fonte externa. Se ambos estiverem conectados, a preferência será para fonte externa. Acompanhe as fotos seguintes e veja como reconhecer esta situação.

No Arduino original a seleção da fonte de energia é automática.

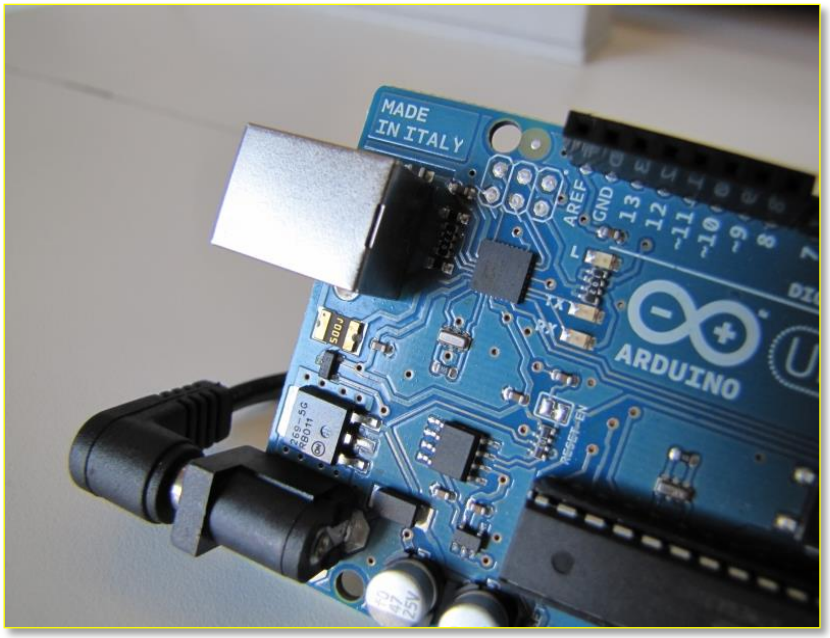

Figura 2-3 – placa com seleção automática de fonte de energia

Em Arduinos de outros fornecedores pode ser necessário a seleção manual da fonte de energia. Compare as duas fotos. Notou a inscrição USB e EXT na placa? A seleção vai indicar se a energia vem da porta USB ou de uma fonte externa.

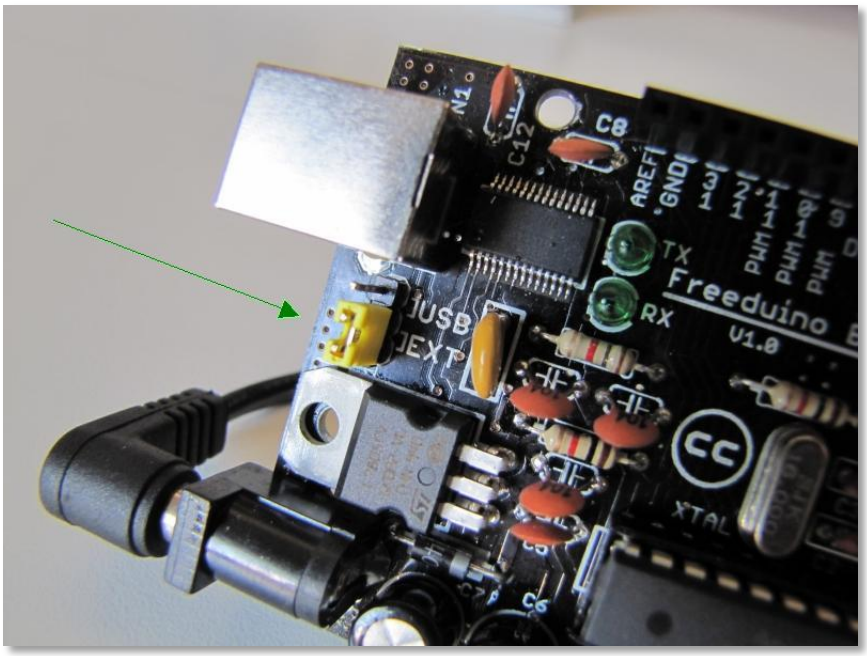

Figura 2-4 – placa com seleção manual de fonte de energia

E por falar em fonte externa, esta poderá ser de uma bateria de 9V ou de um **transformador** entre 9V e 12V de corrente contínua. O transformador é responsável por converter (reduzir) a tensão de 110V ou 220V, que está presente nas nossas residências, para uma tensão compatível com a entrada na placa do Arduino.

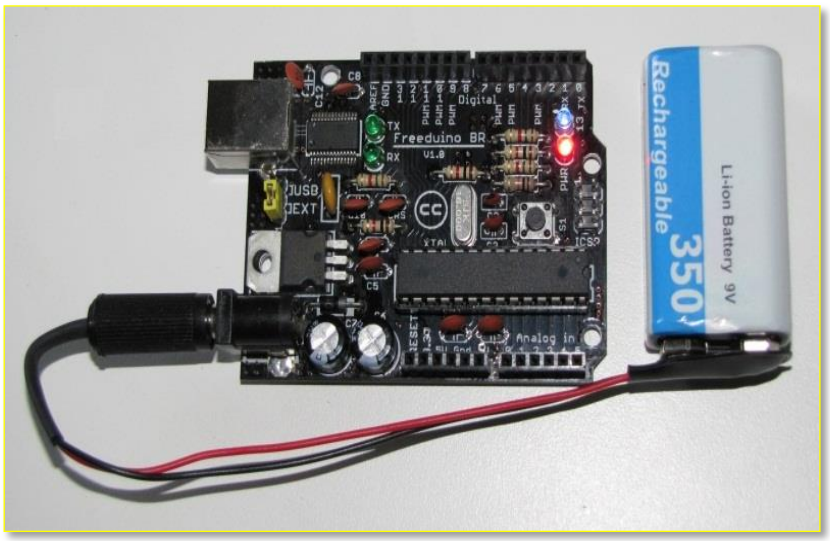

Figura 2-5 – alimentação por uma bateria de 9V

Figura 2-6 - alimentação por uma fonte com transformador

Independente da sua opção, é importantíssimo saber que, a tensão de uma fonte externa seja de 9V ou 12V, será reduzida para 5V que é a tensão efetivamente aceita pelo Arduino. Lembra quando falamos sobre o cabo USB? Que havia uma linha de 5V presente no cabo? Pois bem, este é o valor de tensão presente em grande parte dos chamados circuitos digitais, que incluem o seu computador. Então, apesar de termos uma faixa aceitável na entrada da placa (entre 9V e 12V), o que realmente é usado é 5V. Existe um componente responsável por esta redução, que esta logo ao lado da entrada de fonte externa. Veja no detalhe da foto.

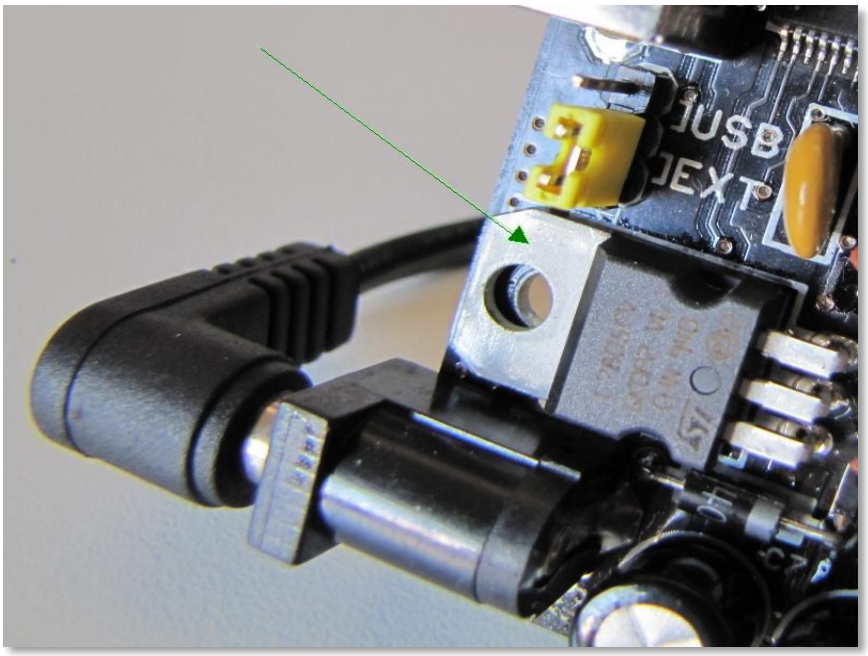

Figura 2-7 – regulador de tensão

2.1.4 Resistência

Como vimos anteriormente, os condutores permitem a passagem da corrente elétrica. Porém, não temos um condutor ideal. Um bom condutor, como o cobre, oferece uma baixa resistência (oposição) à passagem da corrente elétrica. A **resistência elétrica** depende do tipo de material e de suas dimensões, entre outros fatores. Quanto maior a resistência encontrada no material condutor, menor será a intensidade da corrente elétrica.

A unidade de medida é o **ohm (Ω)**, homenagem ao físico alemão Georg Simon Ohm. Ohm relacionou as três grandezas (corrente, tensão e resistência), ou seja, a relação entre o valor da tensão (V) aplicada a um material e o valor da corrente (I) que percorre este material representa a sua resistência elétrica (R), resultando na primeira "Lei de Ohm". Mais adiante usaremos esta fórmula.

$$R = \frac{V}{I}$$

Pronto!Estes são os principais conceitos sobre eletricidade. Nos próximos módulos, a interação entre os componentes, a placa Arduino e software de desenvolvimento serão apresentados sempre com uma atividade prática.

3 Placa para testes (prototipagem)

Vamos usar em nossas atividades, uma placa para testes conhecida como ***protoboard*** ou ***breadboard*** ou ainda de **matriz de contatos**.

Este recurso é muito útil para iniciantes e até mesmo para projetistas experientes, pois facilita a montagem dos circuitos eletrônicos sem a necessidade de solda. Desta forma você pode reutilizar estes componentes sem a preocupação de danificá-los. Naturalmente para projetos permanentes este não é um recurso recomendável.

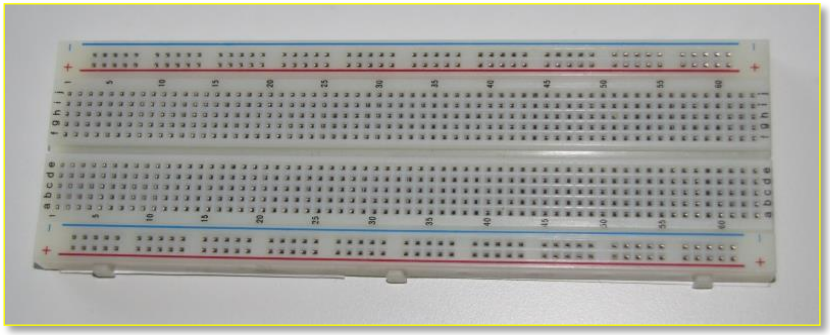

Figura 3-1 – *protoboard*

Figura 3-2 – como os contatos estão interligados

Nos furos centrais, que são as linhas numeradas de 1 a 28 neste exemplo, estas linhas estão interligadas ou em curto e nas laterais as linhas também estão interligadas, mas toda a linha.

No próximo módulo serão apresentadas várias atividades práticas para fixação do conteúdo e diversão ☺. Os esquemas elétricos/eletrônicos (representação gráfica da interligação dos componentes) foram desenvolvidos com um software, *open source*, de nome **Fritzing**.

Na área de downloads do site EADuino (http://www.eaduino.com.br/recursos/downloads/) temos um tutorial dando os primeiros passos neste software. Quando puder não deixe de ler e praticar.

4 Atividades

Ao escrever este material pensamos em algo como "aprender fazendo". Os conceitos vão sendo apresentados e associados a uma atividade prática. Acreditamos que as pessoas, que não conhecem a plataforma Arduino e/ou não são da área de eletrônica e computação, terão a oportunidade de aprender e usar esta tecnologia. Ao longo das atividades propostas, vamos suprimindo algumas telas e instruções que já foram explicadas.

Será necessário providenciar o seguinte material para poder acompanhar todas as atividades práticas propostas:

- um Arduino compatível com a versão Duemilanove ou Uno
- um cabo USB do tipo A / B
- uma *breadboard/protoboard* pequena
- diversos fios jumpers
- cinco resistores de 220 ohms
- um resistor de 10K ohms
- um resistor de 100K ohms
- um NTC 10K ohms
- um LDR de 10mm
- dois LEDs verdes (qualquer tamanho)
- dois LEDs vermelhos (qualquer tamanho)
- um LED amarelo (qualquer tamanho)
- uma chave táctil

Ainda não possui uma placa Arduino e componentes eletrônicos? Visite:

- http://www.eaduino.com.br/onde-comprar/

Divirta-se!

4.1 Atividade 1 - Acendendo um LED

Nesta atividade você aprenderá a reconhecer e usar os seguintes componentes eletrônicos:

- o resistor;
- o diodo emissor de luz (LED).

4.1.1 Resistor

O resistor é o componente utilizado para limitar a passagem da corrente elétrica. Os resistores são feitos com material condutor de alta resistividade elétrica e dissipam esta energia elétrica na forma de calor. A unidade de medida é o ohm (**Ω**).

Os resistores, de uso geral, são pequenos e os valores destes componentes estão codificados na forma de anéis coloridos.

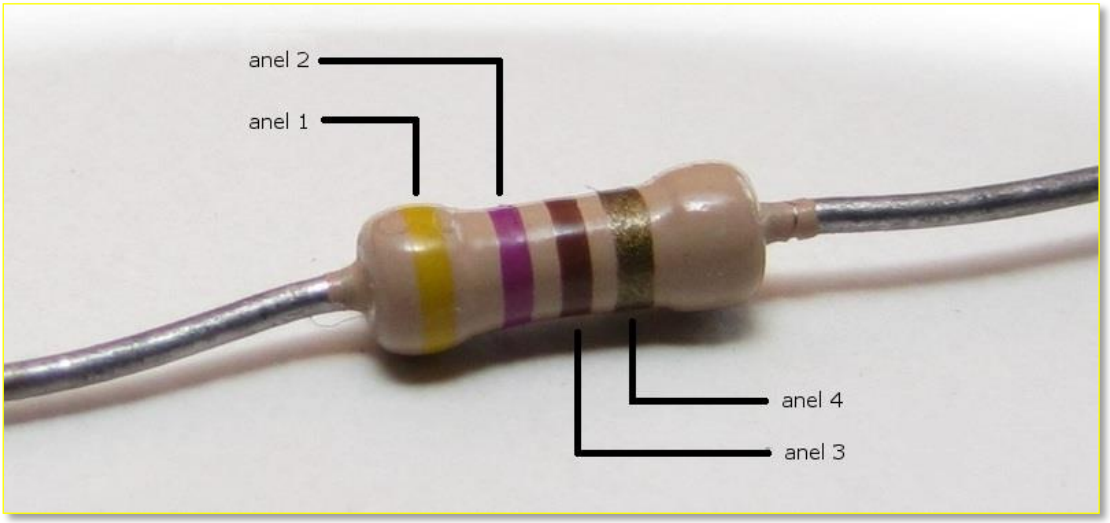

Figura 4-1 – o resistor

O primeiro, segundo e terceiro anéis são os números que indicam o valor do componente. O quarto anel indica o fator (percentual) de tolerância do componente quanto ao seu valor. Isto quer dizer que quanto maior for o fator, menor será a sua precisão.

A leitura é feita segundo a tabela a seguir:

Cor do anel	Anel 1	Anel 2	Anel 3 (multiplicador)	Anel 4 (tolerância)
Preto	-	0	1	-
Marrom	1	1	10	1%
Vermelho	2	2	100	2%
Laranja	3	3	1000	3%
Amarelo	4	4	10000	4%
Verde	5	5	100000	
Azul	6	6	10000000	
Violeta	7	7	-	
Cinza	8	8	-	
Branco	9	9	-	
Prata	-	-	0,01	10%
Dourado	-	-	0,1	5%

Com base nesta tabela, você conseguiria dizer qual é o valor/tolerância deste resistor?

Anel 1 = amarelo
Anel 2 = violeta
Anel 3 = marrom
Anel 4 = dourado

Veja na página seguinte.

Resultado: 470Ω com 5% de tolerância, quer dizer, pode variar para cima ou para baixo em 5% o valor da resistência.

Resistores que possuem valores muito altos são expressos pelos seus múltiplos. Os múltiplos do ohm mais encontrados nos circuitos são: **quiloohm (KΩ)** e **megaohm (MΩ).**

Então, um resistor de 1000 Ω é indicado como 1K Ω. É muito comum também que um resistor de 4700 Ω seja indicado como 4K7 Ω.

4.1.2 LED

O **LED** (Light Emitting Diode – Diodo Emissor de Luz) é um componente eletrônico que emite luz visível (exceto LEDs infravermelhos e ultravioletas) quando conduz a corrente no sentido direto, isto significa que é um componente que tem que ser ligado corretamente para que funcione.

O terminal positivo é o **anodo** e o terminal negativo é o **catodo**, representado geralmente nos circuitos respectivamente pelas letras A e K.

Os LEDs são produzidos nos mais variados tamanhos e formatos. Os mais comuns são os redondos de 5mm e 3mm. Na foto, temos um não muito comum: o chato.

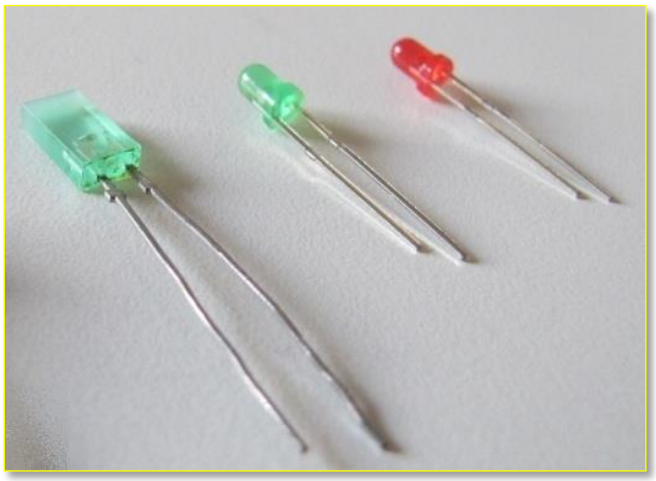

Figura 4-2 – LEDs de tamanho e formato diversos

Nos LEDs, normalmente, podemos identificar visualmente qual é o terminal (*lead*) positivo pelo seu comprimento mais longo que o negativo. Outra maneira, notar que um lado do componente é chanfrado, indicando o terminal negativo.

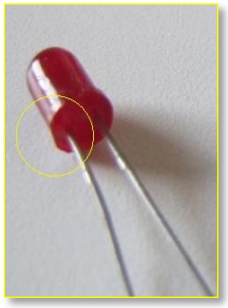

Figura 4-3 – chanfro indicando terminal negativo (catodo)

4.1.3 O resistor como limitador de corrente

Para que um LED comece a conduzir temos que ter uma tensão mínima de aproximadamente 2V. Tensões abaixo deste valor não conseguem fazer o LED funcionar. Quando o LED começa a conduzir, este apresenta uma resistência muito baixa e se não limitarmos a corrente que circula pelo LED, fatalmente o componente será danificado.

A corrente máxima suportada pelos LEDs está em torno de **50mA**. Utilize de **10mA** a **20mA** para uma **operação segura**. Para limitarmos esta corrente usaremos o resistor e o seu valor será obtido por uma fórmula bem simples (lei de Ohm). O resultado será em **ohms**.

Pronto para a primeira atividade?

Verifique se já está com o software Arduino carregado e placa Arduino conectado ao seu computador via cabo USB.

O diagrama abaixo é a representação esquemática do circuito. Se você não está familiarizado com esquemas eletrônicos, com a prática vai ficar mais fácil de interpretá-la. Sempre que tiver dúvidas com a interligação dos componentes, recorra ao diagrama.

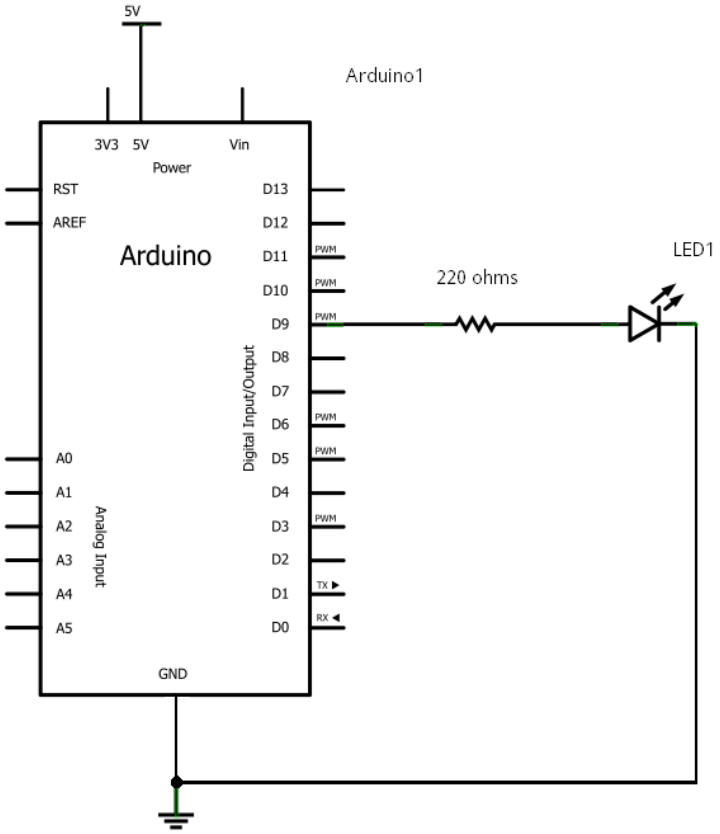

Figura 4-4 – diagrama da atividade 01

Primeiro vamos montar o circuito. Separe os seguintes componentes: um **resistor de 220 ohms** (vermelho, vermelho e marrom) e um **LED vermelho.**

A figura seguinte é a uma placa Arduino conectada a *protoboard* com os respectivos componentes. Tente associar o esquema anterior com o desenho abaixo.

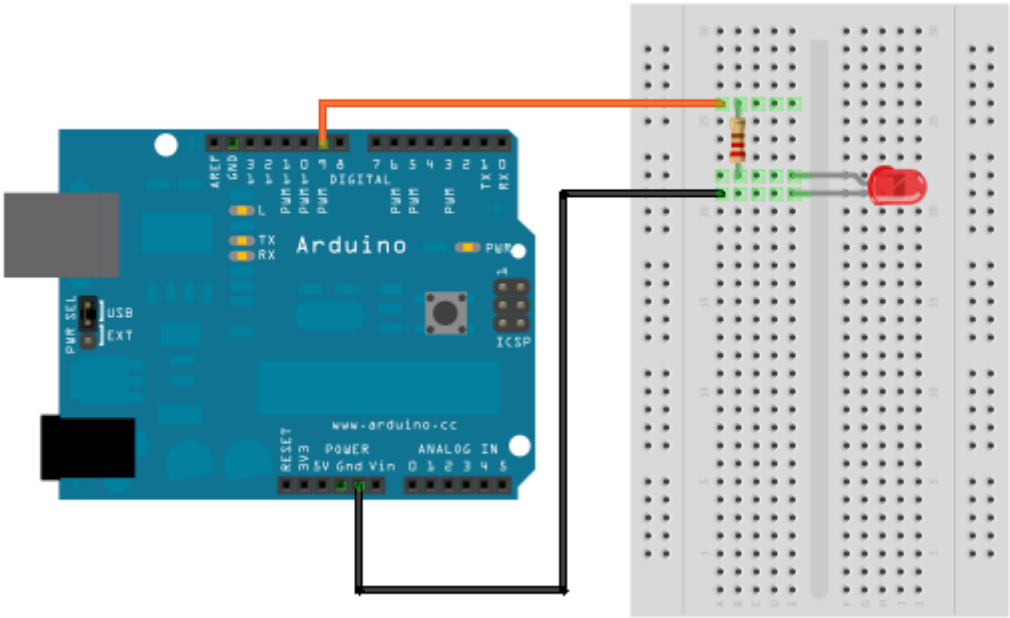

Figura 4-5 – interligando os componentes com a placa Arduino

Tente posicionar a sua placa Arduino e a *protoboard* conforme o desenho acima.

Coloque os componentes como no desenho. O resistor não tem lado certo para ser ligado. O terminal mais longo do LED é ligado na mesma trilha da outra extremidade do resistor. Por fim ligue os fios jumpers:

- uma ponta do fio laranja vai no pino 9 da placa e a outra ponta na extremidade livre do resistor.

- uma ponta do fio preto vai no pino Gnd da placa e a outra ponta na extremidade livre do LED.

Agora iremos criar o programa que fará o LED acender.

Inicie o ambiente de desenvolvimento do Arduino, que a partir de agora chamaremos de IDE (*Integrated Development Environment* - ambiente integrado de desenvolvimento). A IDE do Arduino permite editar, verificar, compilar (traduzir o programa para um formato que o Arduino entenda) e transferir o programa (*sketch*) já verificado para a placa.

4.1.4 Estrutura de um sketch

Toda linguagem de programação, inclusive a linguagem do Arduino, possui regras na escrita (codificação) que devem ser respeitadas. Muita atenção deve ser dada se a palavra reservada da linguagem possuir caracteres maiúsculos ou minúsculos. Escreva exatamente como descrito.

Como já falamos, no universo Arduino, um programa é chamado de sketch. Todo sketch possui uma estrutura mínima para poder funcionar. Devemos inserir dois blocos ou funções: **setup()** e **loop()**.

4.1.5 setup()

Este bloco é executado logo após o inicio do sketch. O que estiver no bloco setup(), é executado apenas uma vez, ou toda vez que o Arduino for reiniciado (pressionando um botão específico na placa ou quando da falta de energia).

4.1.6 loop()

Após a execução do bloco setup(), o controle do programa vai para o bloco loop() e como o nome sugere, fica “eternamente” neste bloco. Aqui, efetivamente, instruímos o Arduino a fazer o que queremos.

Pronto para iniciar?

Com a IDE do Arduino aberta, digite o código a seguir, exatamente como escrito, respeitando caracteres minúsculos ou maiúsculos e sinais de pontuação. O texto, para facilitar a leitura, contém cores e caracteres formatados (negrito e/ou itálico). Não leve isto em consideração na hora de digitar. Após a execução do *sketch* vamos explicar detalhadamente cada linha.

```
/*
 Atividade 01
*/

void setup() {
  // configura o pino 9 como saida
  pinMode(9, OUTPUT);
}

void loop() {
  // envia nivel alto para o pino 9
  digitalWrite(9, HIGH);
}
```

Vamos verificar se não há erros de digitação. Na barra de botões da IDE, clique com o botão esquerdo do mouse no ícone indicado na figura abaixo (note que ao posicionar o mouse neste ícone, você verá a palavra **Verify** a direita do último botão).

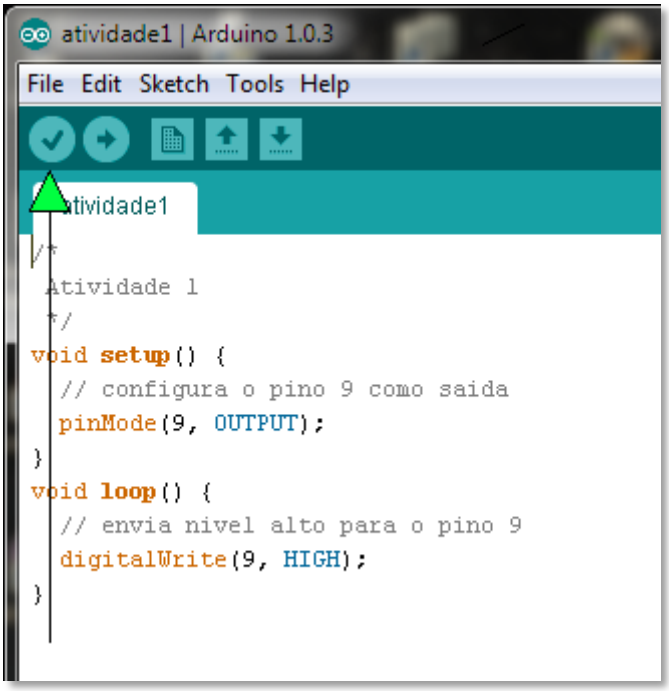

Figura 4-6 – botão de verificação

Se não houver erros, no rodapé da tela do IDE aparecerá **Done compiling**.

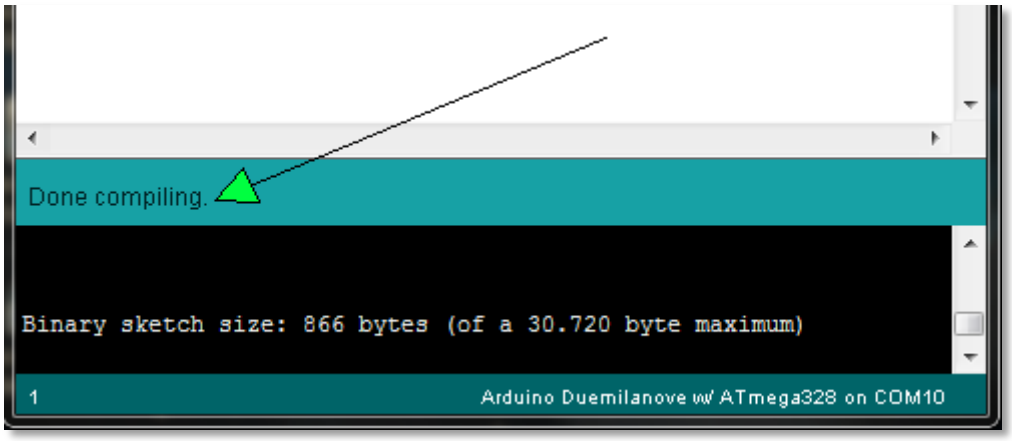

Figura 4-7 – área de mensagens: compilação feita

Caso apareça algum erro, reveja o sketch digitado.

Importante! Este botão apenas faz a verificação. O sketch até este momento ainda não foi transferido para a placa.

Para transferir, clique com o botão esquerdo do mouse no ícone indicado na figura abaixo.

Figura 4-8 – botão de transferência

No rodapé temos a informação que a aplicação foi transferida: **Done uploading**.

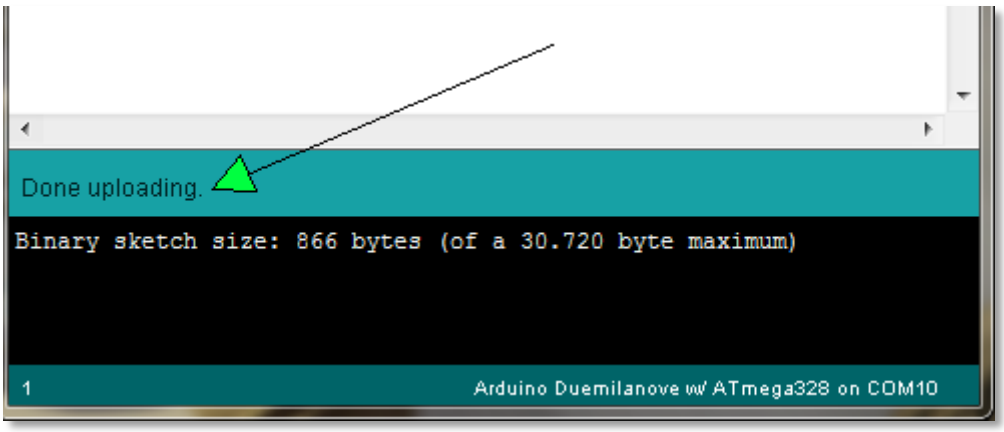

Figura 4-9 – área de mensagens: transferência feita

O resultado esperado é que um LED da placa Arduino fique piscando.

Vamos salvar o *sketch*. Acesse o menu **File** e selecione **Save As...**, escolha uma pasta qualquer no seu computador e dê o nome de **atividade01.** Em seguida clique no botão **Salvar** da tela de diálogo do Windows.

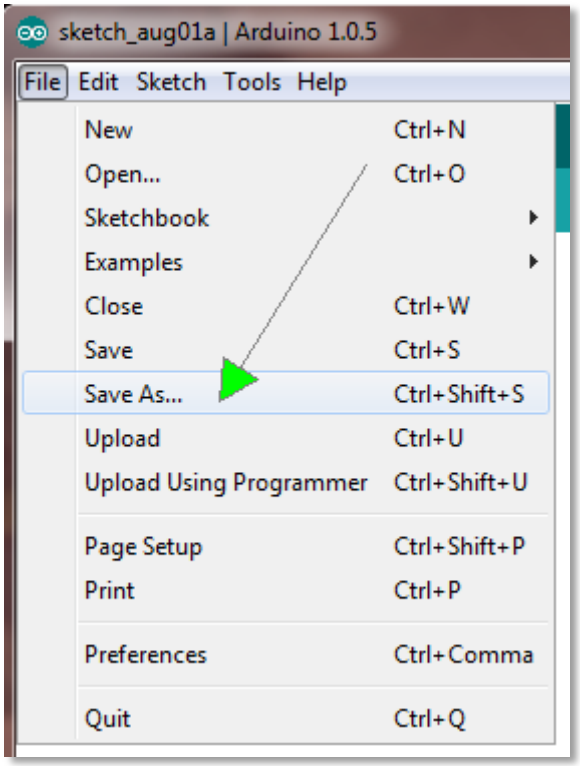

Figura 4-10 – opção Salvar Como...(Save As...)

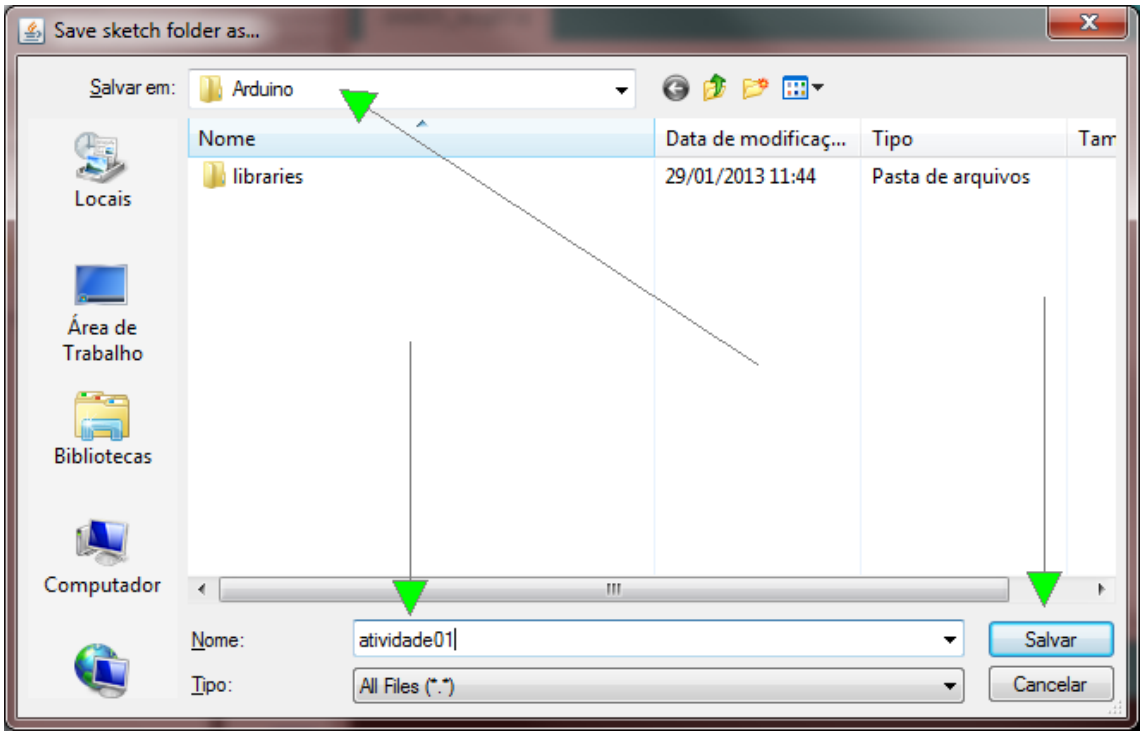

Figura 4-11 – opção Salvar Como...(Save As...)

Importante! o arquivo salvo ficará dentro da pasta seleciona e este terá a extensão **.ino**.

Agora vamos explicar os elementos deste *sketch*. Acompanhe as explicações a seguir com o código que está na tela da IDE.

4.1.7 Comentários

/*...*/
//

Na linguagem Arduino, estes caracteres indicam que o texto que está entre os caracteres /* e */, e após // devem ser ignorados pelo compilador, pois são apenas comentários inseridos no *sketch* e o mais importante é que estes comentários não ocupam espaço na memória do Arduino. Então, use e abuse de comentários. Isto nos ajuda a documentar o programa, fazendo com que qualquer outra pessoa possa entender o propósito do *sketch.*

4.1.8 setup()

void setup() {

}

Este bloco ou função, como comentado anteriormente, é executado apenas uma vez quando o sketch é iniciado. Podemos entendê-lo como um bloco de configuração inicial do nosso hardware. A palavra **void** que antecede o nome da função é obrigatória e não será objeto de estudo neste tutorial introdutório.

Importante! Toda função na linguagem Arduino, começa e termina por abre e fecha chaves.

pinMode(pino , INPUT|OUTPUT)

Este comando indica ao Arduino como queremos que ele atue no nosso circuito. Arduinos Duemilaenove e Uno, e seus compatíveis, utilizam atualmente o microcontrolador ATMEGA328P. Temos a disposição **14** pinos digitais configuráveis como entrada ou saída e **6** pinos analógicos de entrada. Então a declaração: `pinMode(9, OUTPUT);` configura o pino 9 como saída.

Importante! Toda declaração na linguagem Arduino termina com ponto e vírgula.

4.1.9 loop()

void loop() {

}

Esta função executa "eternamente" os comandos que estiverem no bloco. Este laço só é interrompido se houver falta e posterior retorno de energia ou se você pressionar o botão *reset* presente na placa. Este é o bloco que faremos a leitura de sensores, comandaremos luzes, ligaremos motores, etc.

4.1.10 digitalWrite()

digitalWrite(pino , HIGH|LOW)

Esta função envia um nível alto (*high*) ou baixo (*low)* para o pino informado. Toda a lógica que comanda o Arduino e os circuitos digitais é dada através de níveis de tensão que vão indicar nível alto ou baixo. Por exemplo: se enviarmos um nível alto para um pino, teremos neste pino 5V e se enviarmos um nível baixo, teremos 0V. Então a declaração: `digitalWrite(9, HIGH);` envia para o pino 9 uma tensão de 5V. Isto faz com que o nosso LED acenda!

Voltando ao nosso circuito, podemos calcular a corrente (I) que passa pelo nosso LED.

$$R = \frac{V}{I}$$

Ou

$$I = \frac{V}{R}$$

Antes de aplicarmos a fórmula é preciso saber que os LEDs apresentam uma queda de tensão em seus terminais, que variam conforme a cor, tipo e fabricante. De um modo geral, para aplicações com o Arduino, podemos assumir um valor médio de 2V.

Substituindo pelos valores presentes no circuito, temos

$$I = \frac{(5V - 2V)}{220\ \Omega}$$

A corrente que irá passar pelo LED, usando um resistor de **220Ω**, será de 0,013A (ampère) ou 13mA (multiplicamos por 1000 para obter mili ampère).

4.2 Atividade 2 - Pisca LED

Nesta atividade usaremos componentes eletrônicos já conhecidos:

- o resistor;
- o diodo emissor de luz (LED).

Monte o mesmo circuito apresentado na Atividade 1 e digite o *sketch* abaixo.

```
/*
 Atividade 02
*/

int pinoLED = 9;

void setup() {
  // configura o pino como saida
  pinMode(pinoLED, OUTPUT);
}

void loop() {
  // envia nivel alto para o pino
  digitalWrite(pinoLED, HIGH);

  // aguarda 1 segundo
  delay(1000);

  // envia nivel baixo para o pino
  digitalWrite(pinoLED, LOW);

  // aguarda 1 segundo
  delay(1000);
}
```

Transfira o *sketch* para a placa (botão Upload).

Dica: Na atividade anterior solicitamos a IDE fazer a verificação (*Verify*) do sketch e somente depois transferimos para a placa. Com o tempo, você poderá abreviar este procedimento optando apenas pelo *Upload*. Qualquer anormalidade será indicada no rodapé da interface.

Se não houver erros, o LED deverá piscar.

Vamos salvar o sketch. Acesse o menu **File** e selecione **Save As...**, escolha uma pasta qualquer no seu computador e dê o nome de **atividade02**.

Agora vamos explicar os elementos deste *sketch*. Acompanhe as explicações a seguir com o código que está na tela da IDE.

4.2.1 Variáveis

<tipo> <nome> = valor

As variáveis são locais na memória do microcontrolador para guardar dados enquanto o sketch estiver sendo executado. Como o nome diz, podemos alterar este dado na hora que convier. Na linguagem Arduino as variáveis possuem um **tipo**, **nome** e opcionalmente o sinal de **atribuição (=)** e **valor**, e devem ser declaradas antes do seu uso.

Na declaração `int pinoLED = 9;` diz que criamos uma variável de nome pinoLED do tipo int (inteiro), cujo valor é 9.

Variáveis auxiliam muito o programador, permitindo que trabalhemos com dados que são alterados todo momento. Exemplo: um sensor de temperatura ou um sensor de luz envia um dado através de um pino para o programa, informando o seu valor. Este valor é armazenado em uma variável para posteriormente fazermos um cálculo, acionarmos um outro dispositivo, imprimirmos o valor na tela do computador, etc.

Algumas sugestões com relação ao nome:

- Não use apenas um caractere para nomear uma variável. Exemplo: x
- Dê nomes que façam sentido ao projeto ou o trecho de código que está sendo desenvolvido. Exemplo: apagarLuz
- Cuidado com palavras da língua inglesa. Você pode, sem querer, usar um nome reservado para a linguagem Arduino.

Quanto ao tipo, apresentaremos os principais tipos (que serão usados nesta apostila).

4.2.2 int

Um tipo int (inteiro) pode manipular números na faixa de -32.768 até 32.767. No sketch, a declaração está armazenando o inteiro 9 na variável pinoLED.

No momento você precisa saber de mais uma coisa. Notou que a declaração da variável foi feito fora do bloco setup() e loop()?

Na linguagem Arduino, temos o conceito de **escopo** (abrangência) de variáveis. De uma maneira bem simples, quando declaramos variáveis **fora de qualquer bloco**, estas são chamadas **globais** e se declaradas **dentro de algum bloco**, de **locais**. Quando uma variável global é declarada, esta pode ser manipulada em qualquer parte do sketch, ao contrário das locais, que são apenas "vistas" dentro do bloco que a criou.

Na declaração `pinMode(pinoLED, OUTPUT);` pinoLED é uma variável global e por isto pode ser usada dentro do bloco setup(). Durante a execução, o Arduino substitui o nome desta variável pelo seu valor, no caso 9.

4.2.3 delay()

delay(tempo)

Esta função pausa a execução do sketch durante o tempo informado. O parâmetro tempo é em mili segundos, ou seja, a declaração `delay(1000);` pausa a execução por 1 segundo.

Concluindo. No bloco loop(), o Arduino envia 5V para o pino 9, aguarda 1 segundo e envia 0V para o pino 9. Com isto temos o efeito de pisca-pisca.

4.3 Atividade 3 - Controle de Luminosidade de LEDs

Nesta atividade usaremos componentes eletrônicos já conhecidos:

- o resistor;
- o diodo emissor de luz (LED).

O diagrama abaixo é a representação esquemática do circuito.

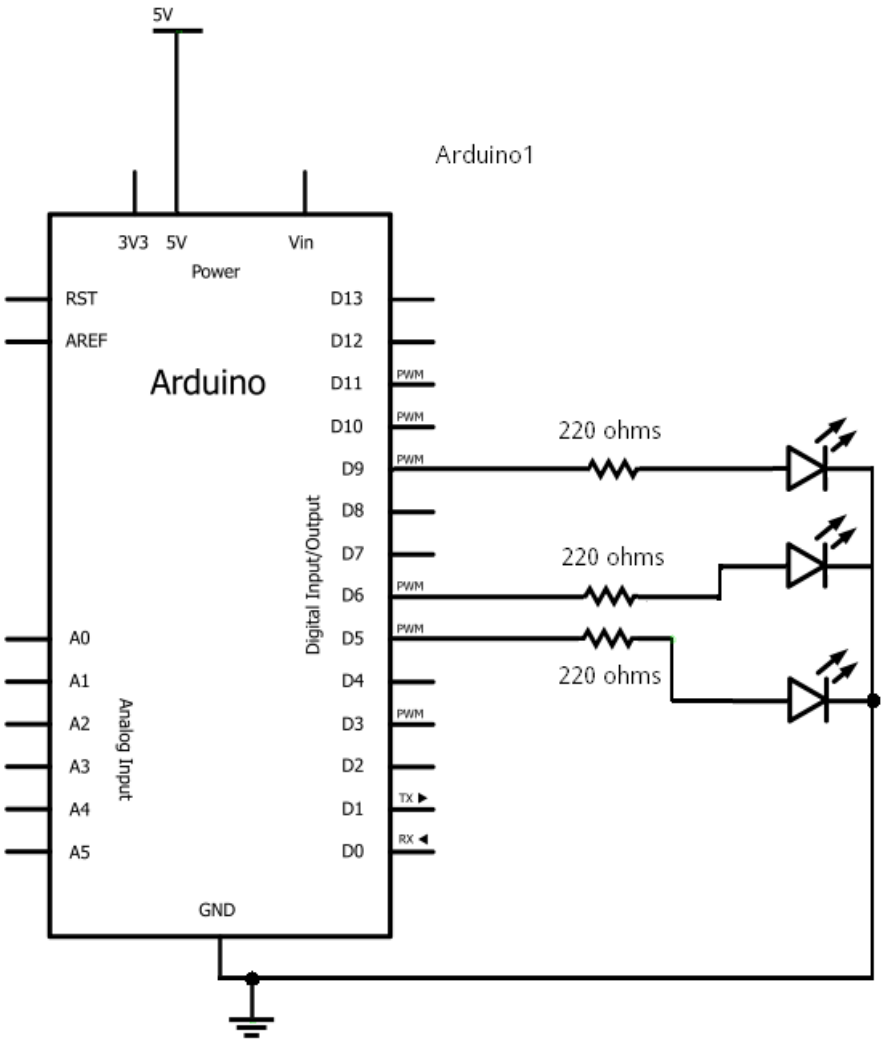

Figura 4-12 – diagrama atividade 03

Separe os seguintes componentes:

- três resistores de 220 ohms
- um LED vermelho
- um LED amarelo
- um LED verde

A figura seguinte é a placa Arduino conectada a *protoboard* com os respectivos componentes. Tente associar o esquema anterior com o desenho abaixo.

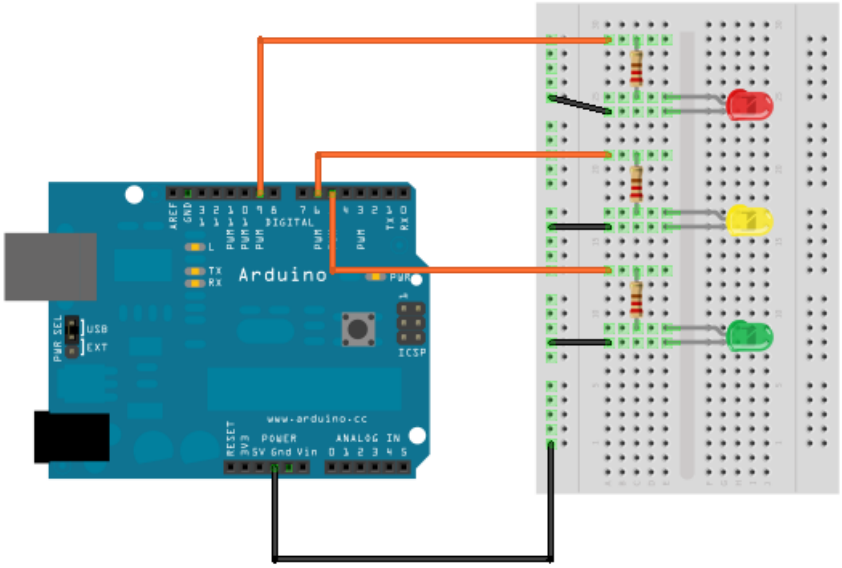

Figura 4-13 – interligando os componentes com a placa Arduino

Digite o sketch abaixo e transfira para a placa Arduino.

```
/*
 Atividade 03
*/

int pinoLEDvermelho = 5;
int pinoLEDamarelo  = 6;
int pinoLEDverde    = 9;

int brilho;

void setup()  {
  pinMode(pinoLEDvermelho, OUTPUT);
  pinMode(pinoLEDamarelo, OUTPUT);
  pinMode(pinoLEDverde, OUTPUT);
}

void loop()  {

  // aumenta o brilho dos LEDs, incrementando de 5 em 5 passos

  for( brilho = 0 ; brilho <= 255; brilho = brilho + 5) {
    analogWrite(pinoLEDvermelho, brilho);
    analogWrite(pinoLEDamarelo, brilho);
    analogWrite(pinoLEDverde, brilho);
    // aguarda 30 milisegundos
    delay(30);
  }

  // diminui o brilho dos LEDs, decrementando de 5 em 5 passos

  for( brilho = 255 ; brilho >= 0; brilho = brilho - 5) {
    analogWrite(pinoLEDvermelho, brilho);
    analogWrite(pinoLEDamarelo, brilho);
    analogWrite(pinoLEDverde, brilho);
```

```
    // aguarda 30 milisegundos
    delay(30);
  }
}
```

Salve o *sketch* com o nome **atividade03**. O resultado esperado é aumentar e diminuir o brilho dos três LEDs de forma gradativa**.**

Agora vamos explicar os elementos deste *sketch*. Acompanhe as explicações a seguir com o código que está na tela da IDE.

Após a declaração das variáveis globais, o bloco setup() configura os pinos representados pelas variáveis como saída.

No bloco loop(), usamos uma estrutura de repetição que farão os três LEDs aumentarem de brilho e em seguida, uma outra estrutura de repetição farão os três LEDs diminuírem o brilho.

4.3.1 for()

for(início ; condição ; incremento) {

}

A estrutura **for** é uma das mais versáteis da linguagem Arduino. As declarações (dentro deste bloco) são repetidas até que a condição estabelecida seja verdadeira.

No nosso sketch,

```
for( brilho = 0 ; brilho <= 255; brilho = brilho + 5) {

}
```

Atribui o valor 0 para a variável brilho e enquanto a variável é menor ou igual ao valor 255, o que está dentro do bloco é executado. Ao encontrar fecha chaves, a variável brilho é incrementada (o seu valor é acrescido) de cinco em cinco. Isto é repetido até que a variável brilho atinja o valor de 255.

E a estrutura,

```
for( brilho = 255 ; brilho >= 0; brilho = brilho - 5) {

}
```

Faz o inverso da anterior, a cada passagem a variável brilho vai sendo decrementada (o seu valor é diminuído) de cinco em cinco até que a variável brilho atinja o valor 0.

4.3.2 Operadores

Nesta estrutura também vimos algo novo, o operador relacional <= (menor ou igual). A linguagem Arduino possui os seguintes operadores relacionais:

- `==` (igual a)
- `!=` (diferente)
- `<` (menor que)
- `>` (maior que)
- `<=` (menor ou igual a)
- `>=` (maior ou igual a)

Estes operadores permitem fazermos comparações entre uma variável e um valor ou entre variáveis.

Importante! Muito cuidado com o operador `==` (igual a). Não confunda com o sinal `=` que indica atribuição. Este é um tipo de engano que podemos levar horas para descobrir a razão do programa não funcionar como esperado.

Aplicamos também os operadores aritméticos + (adição) e – (subtração). A linguagem Arduino possui os seguintes operadores aritméticos:

- `+` (adição)
- `-` (subtração)
- `*` (multiplicação)
- `/` (divisão)
- `%` (módulo - resto da divisão)

4.3.3 analogWrite()

analogWrite(pino , valor PWM)

Nas atividades anteriores enviamos um sinal digital (HIGH ou LOW) para um pino através da função digitalWrite(). Pois bem, esta função apenas envia um de dois estados possíveis: alto ou baixo que representam respectivamente 5V ou 0V. Porém, nesta atividade, usamos um efeito conhecido como *fade in* (a intensidade do brilho vai aumentando) e *fade out* (a intensidade do brilho vai diminuindo) e neste caso, não poderíamos usar um sinal digital, pois não conseguiríamos o "meio termo".

Nestes casos onde precisamos enviar um sinal gradativo para o pino, temos a função analogWrite(). Esta função utiliza uma técnica conhecida como PWM (pulse width modulation – modulação por largura de pulso). O importante é saber por agora que o sinal PWM varia de 0 a 255, sendo que 0 corresponde a 0V e 255 a 5V. Qualquer valor intermediário vai gerar uma tensão proporcional a este valor.

Exemplos:

Valor PWM	Volts
0	0
64	1,25
127	2,5
191	3,75
255	5

No Arduino, não são em todos os pinos que podemos usar o recurso PWM. Nos Arduinos Duemilanove, UNO e compatíveis, somente nos pinos digitais 3, 5, 6, 9, 10 e 11, temos esta função.

Figura 4-14 – posição dos pinos PWM na placa Arduino Uno

Então, nas declarações

```
analogWrite(pinoLEDvermelho, brilho);
analogWrite(pinoLEDamarelo, brilho);
analogWrite(pinoLEDverde, brilho);
```

enviamos para os pinos representados pelas variáveis **pinoLEDvermelho, pinoLEDamarelo** e **pinoLEDverde**, o valor contido na variável **brilho**, que representa o valor PWM que queremos "escrever" no pino.

4.4 Atividade 4 - Ligando/Desligando um LED através de uma chave

Nesta atividade usaremos um novo componente:

- chave táctil.

E os já conhecidos:

- o resistor;
- o LED.

4.4.1 Chave Táctil

É um tipo de chave que, enquanto o botão estiver pressionado, o circuito estará ativo e ao soltar o botão, o circuito é interrompido, ou seja, este tipo de chave não tem trava.

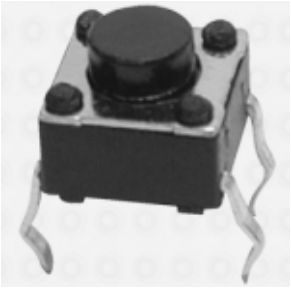

Figura 4-15 – chave táctil

O diagrama abaixo é a representação esquemática do circuito.

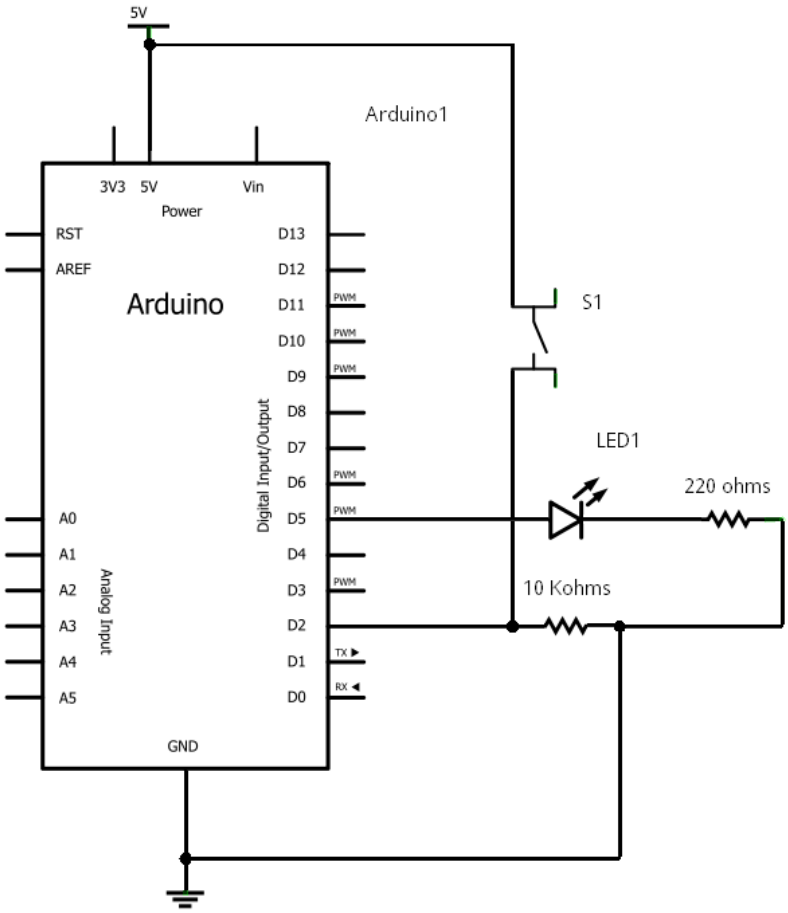

Figura 4-16 – diagrama atividade 04

Separe os seguintes componentes:

- um resistor de 220 ohms
- um resistor de 10K ohms
- um LED verde
- uma chave táctil

A figura seguinte é a placa Arduino conectada a *protoboard* com os respectivos componentes. Tente associar o esquema anterior com o desenho abaixo.

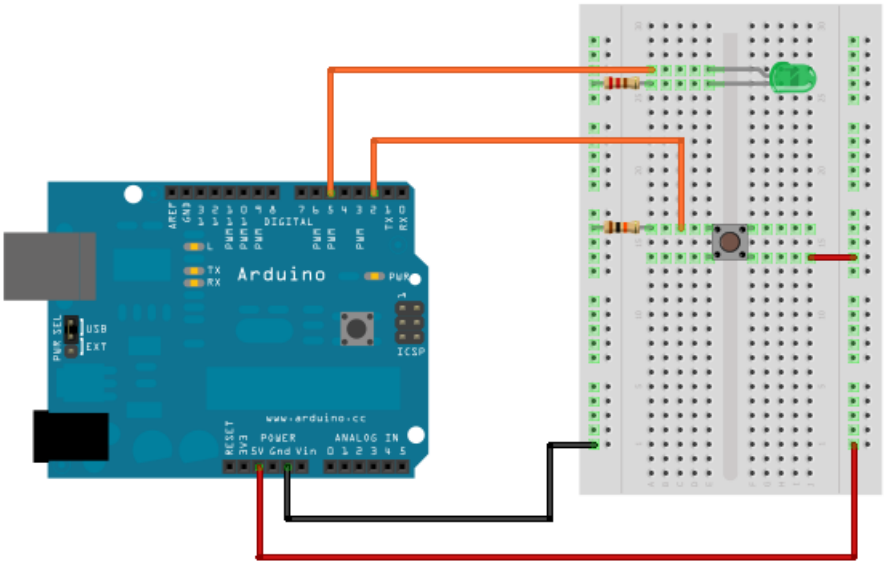

Figura 4-17 – interligando os componentes com a placa Arduino

Digite o sketch abaixo e transfira para a placa Arduino.

```
/*
 Atividade 04
*/

int pinoBotao = 2;
int pinoLED   = 5;

// variavel para ler o estado do botao (chave tactil)
int estadoBotao = 0;

void setup() {
  // configura o pino como saida
  pinMode(pinoLED, OUTPUT);
  // configura o pino como entrada
  pinMode(pinoBotao, INPUT);
}

void loop(){
  // le o estado do botao
  estadoBotao = digitalRead(pinoBotao);

  // verifica se botao foi pressionado
  if (estadoBotao == HIGH) {
    // se sim, acende o LED
    digitalWrite(pinoLED, HIGH);
  }
  else {
```

```
    // caso contrario, apaga o LED
    digitalWrite(pinoLED, LOW);
  }
}
```

Salve o sketch com o nome **atividade04**. O resultado esperado é que ao pressionar o botão, o LED se acenderá.

Após a declaração das variáveis globais, o bloco setup() configura os pinos representados pelas variáveis como saída para o LED e como entrada para o botão.

No bloco loop(), usamos uma estrutura condicional que fará o teste se o botão foi pressionado. Se sim, o LED vai acender.

4.4.2 digitalRead()

digitalRead(pino)

Esta função lê o valor presente no pino indicado. A declaração `estadoBotao = digitalRead(pinoBotao);` armazena na variável **pinoBotao** um de dois estados possíveis: **HIGH** (5V) ou **LOW** (0V).

4.4.3 if()

if(condição verdadeira) {

}
else {

}

Esta estrutura avalia a condição estabelecida no comando. Caso ela seja verdadeira, isto é, se o resultado da avaliação atender a condição colocada por você, as declarações que estiverem dentro do bloco serão executadas. A estrutura **if()** possui um elemento auxiliar, opcional, que é a condição contrária a avaliação feita pelo if(), ou seja, se a condição for falsa as declarações que estiverem dentro do bloco **else** serão executadas.

Então,

```
  if (estadoBotao == HIGH) {
    // se sim, acende o LED
    digitalWrite(pinoLED, HIGH);
  }
  else {
    // caso contrario, apaga o LED
    digitalWrite(pinoLED, LOW);
  }
```

testa se a variável **estadoBotao** é igual a HIGH. Se sim, coloca nível alto no pino representado pela variável **pinoLED**, caso contrário, coloca nível baixo no pino.

Importante! Atenção mais uma vez ao sinal == que representa o operador de comparação.

4.4.4 Resistores pull-up / pull-down

Quando você faz a leitura de um pino e neste pino não há nada conectado, ou seja, não há um sinal alto ou baixo definido, o microcontrolador fica em um estado conhecido como flutuante. Nestes casos, para garantir um correto estado na entrada do pino usamos um resistor, que dependendo da posição em que é ligado é chamado de pull-up ou pull-down.

Este resistor tem a função de limitar a corrente que flui pelo circuito.

No diagrama abaixo temos um resistor na configuração pull-up.

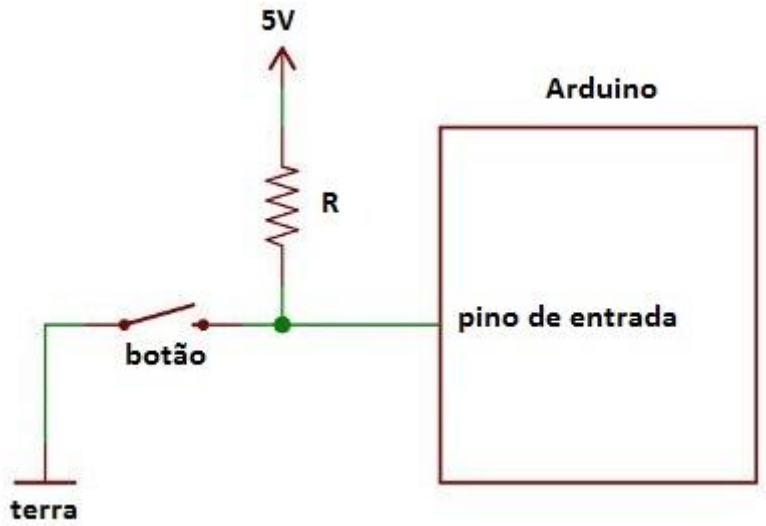

Figura 4-18 – resistor pull-up

Sem pressionar o botão, a corrente circula pelo resistor até o pino, garantindo o nível alto (5V). Quando o botão é pressionado, a corrente é desviada para terra e assim garantindo o nível baixo (0V) neste mesmo pino.

E se o resistor não existisse? Veja a figura a seguir.

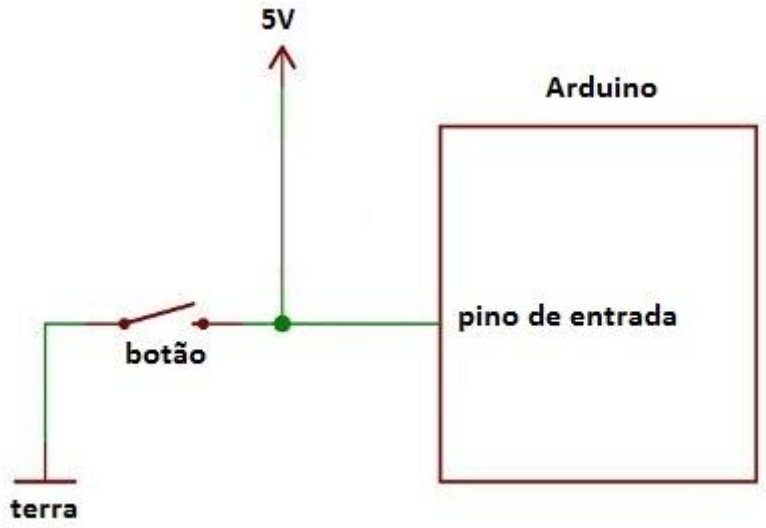

Figura 4-19 – sem o resistor – situação de curto-circuito quando a chave é fechada

Sem pressionar o botão, temos a mesma situação anterior, nível alto (5V). Porém, quando o botão é pressionado, a corrente é desviada para terra sem limite de corrente, ocorrendo a situação de curto-circuito.

Isto pode provocar o esgotamento da fonte de energia (no caso de uma bateria), aumento da temperatura nos condutores ou até mesmo danificar o pino do microcontrolador.

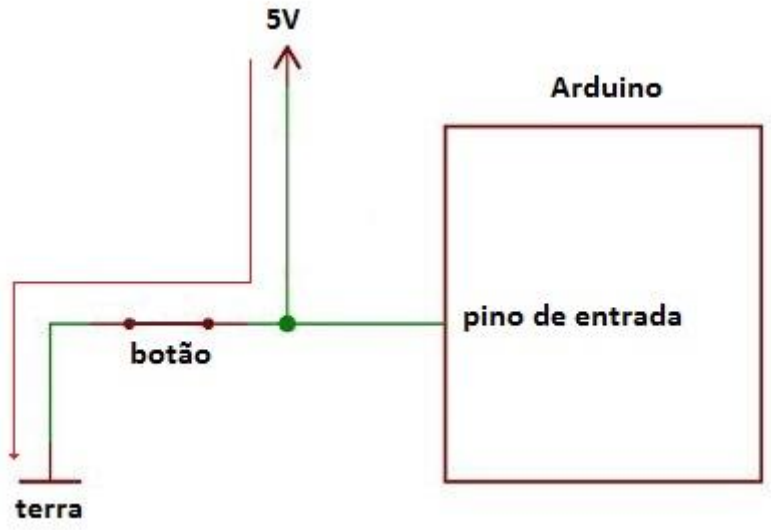

Figura 4-20 – sem o resistor – entrada em curto-circuito

O valor do resistor comumente encontrado nos circuitos é de 10K ohms.

Na nossa atividade, o resistor está configurado com pull-down apenas para simplificar o sketch e ter uma situação próxima do nosso cotidiano. Normalmente pressionamos um botão ou interruptor para acender uma lâmpada, não é?!

Nesta situação temos a seguinte configuração: botão pressionado, LED aceso, caso contrário LED apagado. Em caso de dúvidas, reveja o diagrama desta atividade.

Para fixar melhor este tópico, tente inverter a posição dos componentes. Coloque o resistor com a configuração pull-up e compare com o resultado esperado para esta atividade. O resultado é o inverso, correto?!

4.4.5 Atividade 5 - Controle de luminosidade de LED com LDR

Nesta atividade você aprenderá a reconhecer e usar o seguinte componente eletrônico:

- O LDR.

E os já conhecidos:

- o resistor;
- o LED.

4.4.6 LDR

LDR (Light Dependent Resistor - Resistor Dependente de Luz) é um tipo de resistor cuja resistência varia conforme a intensidade de luz que incide sobre ele. Quanto maior a intensidade, menor a resistência. A resposta do LDR é lenta, ou seja, não é capaz de detectar mudanças rápidas da intensidade da luz.

Figura 4-21 – o LDR

Este componente não requer cuidados especiais. Não há polaridade, isto é, não possui um lado certo para ligá-lo.

O diagrama abaixo é a representação esquemática do circuito.

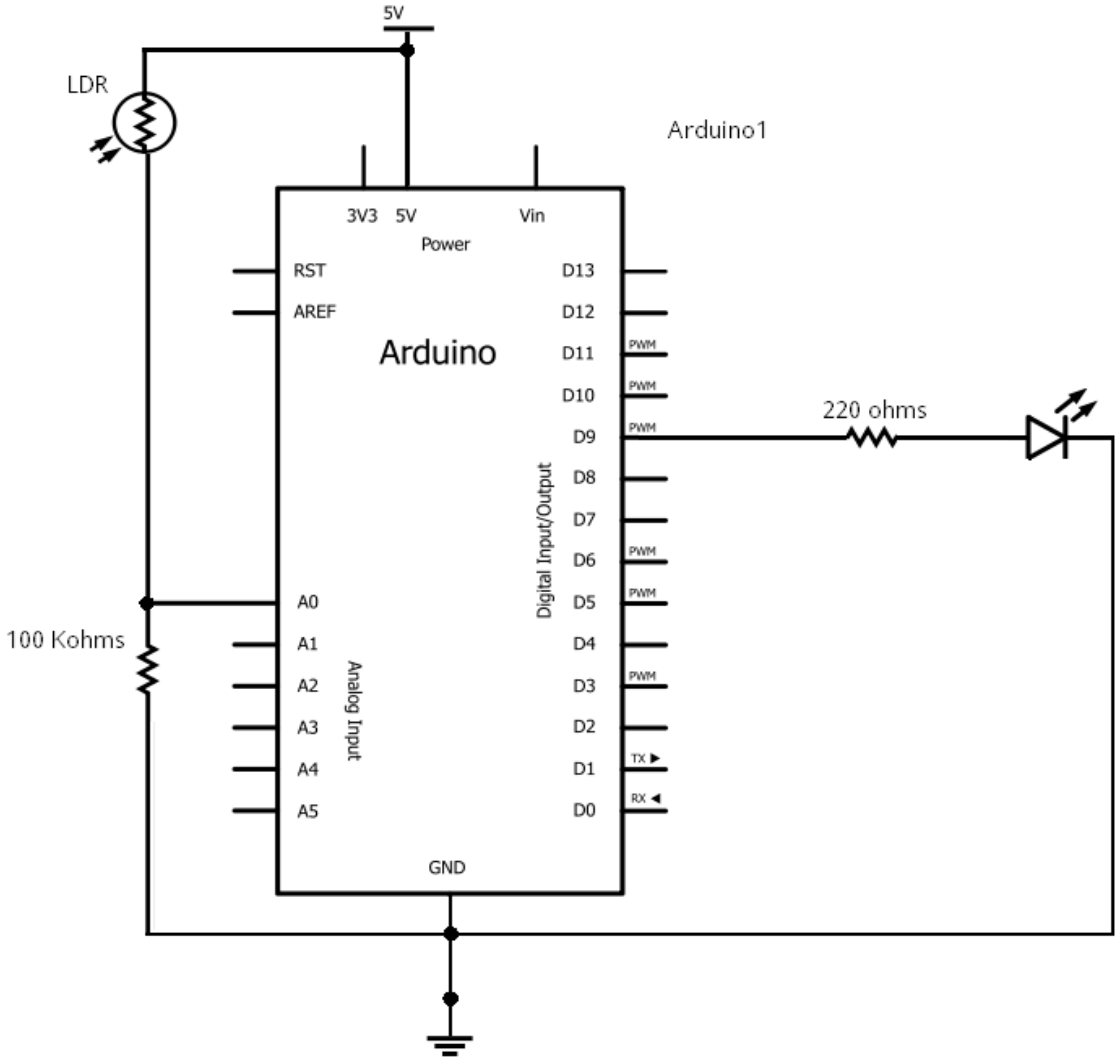

Figura 4-22 – diagrama atividade 05

Separe os seguintes componentes:

- um resistor de 220 ohms
- um resistor de 100K ohms
- um LED vermelho
- um LDR

A figura seguinte é a placa Arduino conectada a *protoboard* com os respectivos componentes. Tente associar o esquema anterior com o desenho abaixo.

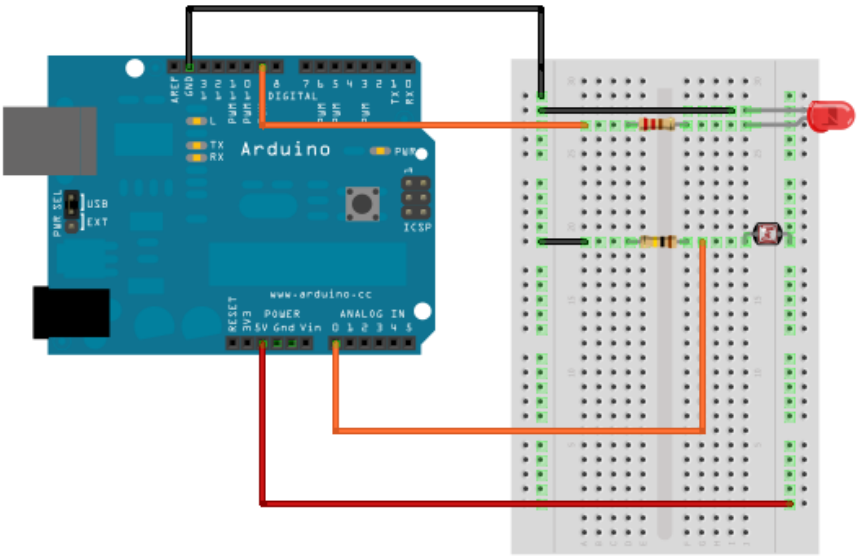

Figura 4-23 – interligando os componentes com a placa Arduino

Digite o sketch abaixo e transfira para a placa Arduino.

```
/*
 Atividade 05
 */

int pinoLED = 9;
int pinoLDR = 0;

float tensaoLida = 0;
int saidaPWM = 0;

void setup()
{
  pinMode(pinoLED, OUTPUT);
  pinMode(pinoLDR, INPUT);
}

void loop()
{
  //--- valor lido sera entre 0 e 1023, cada unidade corresponde a
0.0048828125 volts ou arredondando: 0.0049
  tensaoLida = analogRead( pinoLDR ) * 0.0049;
  //--- dividindo 5 volts por 255 temos 0.0196 volts para cada unidade
PWM
  saidaPWM = 255 - (tensaoLida / 0.0196);
  analogWrite( pinoLED, saidaPWM );
}
```

Salve o sketch com o nome **atividade05**. O resultado esperado será o LED aumentar o brilho ao diminuir a intensidade de luz sobre o LDR.

Vamos ao código...

4.4.7 float

Até o momento usamos apenas o tipo de dado int. Nesta atividade foi necessário usar o tipo **float,** que permite manipular números em ponto flutuante (ponto decimal). Use somente este tipo de dado onde números inteiros não se aplicam.

Você vai entender a necessidade de usar o tipo float na função analogRead().

4.4.8 analogRead()

analogRead(pino)

Esta função lê o valor presente no pino indicado. Diferentemente de digitalRead() que lê somente um valor alto ou baixo, **analogRead()** lê um valor de **0 a 1023** que representa proporcionalmente um valor de tensão, sendo que 0 representa 0V e 1023 representa 5V. Assim, cada unidade representa **0.004882V** (5V dividido por 1024). Em muitas publicações sobre o Arduino, o valor está arredondado para 0.0049V. Para manipular valores como este, temos que usar o tipo de dado float.

Nos Arduinos Duemilanove, UNO e seus compatíveis, temos 6 pinos de entrada analógicos. A resolução (a grandeza que pode ser medida) destas entradas é 10bits. Sabendo que cada bit só pode possuir um de dois estados, alto e baixo, então, 2 elevado a 10 resulta em 1024 (0 a 1023).

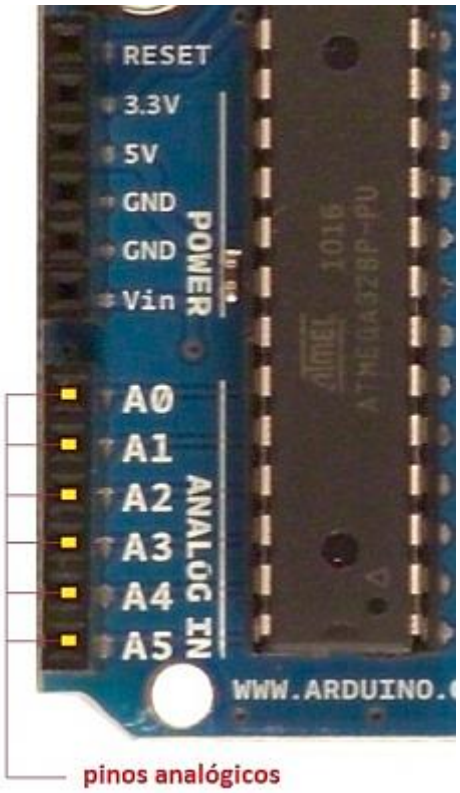

Figura 4-24 – posição dos pinos analógicos

4.5 Atividade 6 - Simulando um semáforo

Nesta atividade usaremos componentes eletrônicos já conhecidos:

- o resistor;
- a chave táctil;
- o diodo emissor de luz (LED).

O diagrama abaixo é a representação esquemática do circuito.

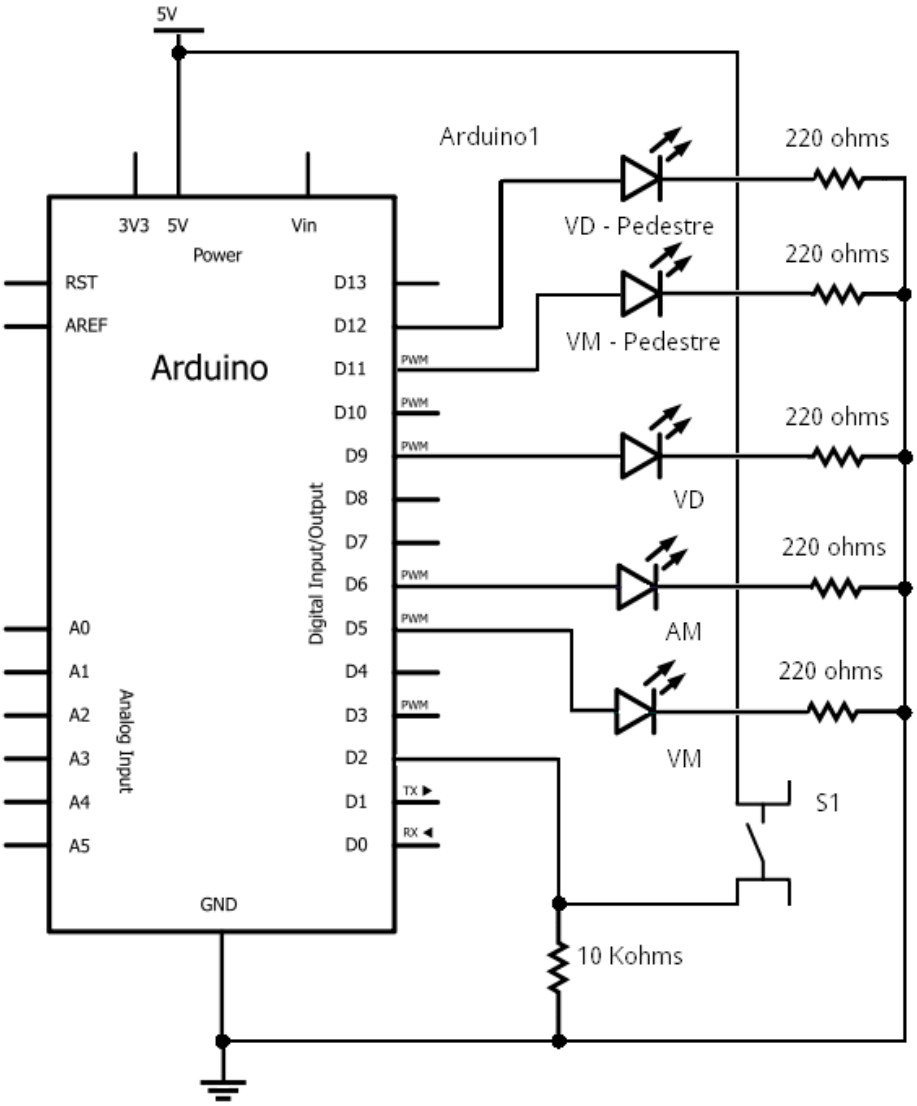

Figura 4-25 – diagrama atividade 06

Separe os seguintes componentes:

- cinco resistores de 220 ohms
- um resistor de 10K ohms
- dois LEDs verdes
- dois LEDS vermelhos
- um LED amarelo
- uma chave táctil

A figura seguinte é a placa Arduino conectada a *protoboard* com os respectivos componentes. Tente associar o esquema anterior com o desenho abaixo. Este circuito é um pouco mais "complexo" em relação às montagens anteriores.

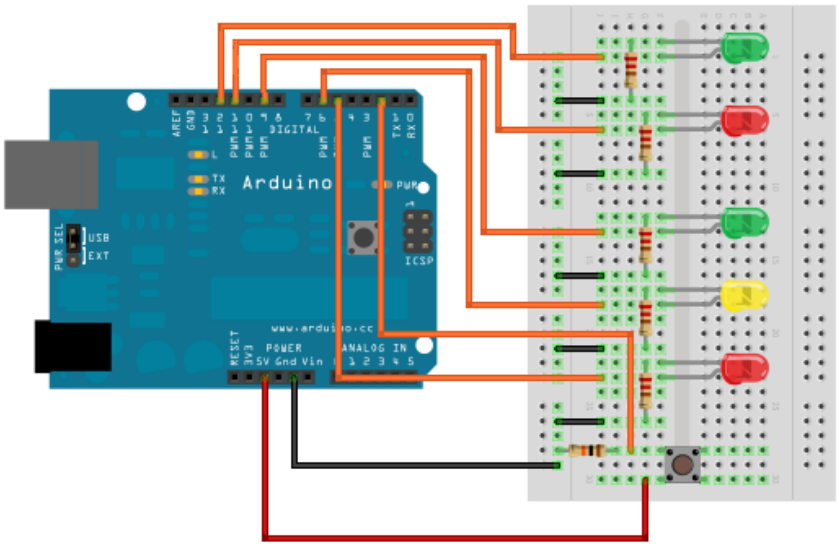

Figura 4-26 – interligando os componentes com a placa Arduino

Digite o sketch abaixo e transfira para a placa Arduino.

```
/*
 Atividade 06
 */

int pinoLEDvermelho          = 5;
int pinoLEDamarelo           = 6;
int pinoLEDverde             = 9;
int pinoLEDvermelho_pedestre = 11;
int pinoLEDverde_pedestre    = 12;
int pinoBotao                = 2;

int estadoBotao = 0;

void setup() {
  // pinos de saida
  pinMode(pinoLEDvermelho, OUTPUT);
  pinMode(pinoLEDamarelo,  OUTPUT);
  pinMode(pinoLEDverde,    OUTPUT);
  pinMode(pinoLEDvermelho_pedestre, OUTPUT);
  pinMode(pinoLEDverde_pedestre,    OUTPUT);
  // pino de entrada
  pinMode(pinoBotao, INPUT);
}

void loop(){
  digitalWrite(pinoLEDvermelho_pedestre, HIGH);
  digitalWrite(pinoLEDverde, HIGH);

  // le o estado do botao
  estadoBotao = digitalRead(pinoBotao);

  // verifica se botao foi pressionado
  if (estadoBotao == HIGH) {
    // semaforo veiculos
    // aguarda 1 minuto (60000 segundos)
    delay(60000);
    // apaga LED verde
    digitalWrite(pinoLEDverde, LOW);
    // acende LED amarelo
    digitalWrite(pinoLEDamarelo, HIGH);
    // aguarda 15 segundos
    delay(15000);
    // apaga LED amarelo
    digitalWrite(pinoLEDamarelo, LOW);
    // acende LED vermelho
    digitalWrite(pinoLEDvermelho, HIGH);
    // semaforo pedestres
    // apaga LED vermelho
    digitalWrite(pinoLEDvermelho_pedestre, LOW);
    // acende LED verde
    digitalWrite(pinoLEDverde_pedestre, HIGH);
    // aguarda 30 segundos
    delay(30000);
    // apaga LED verde
    digitalWrite(pinoLEDverde_pedestre, LOW);
    // pisca LED vermelho 3 vezes
    for( int contador = 0; contador <= 3; contador++ ) {
      digitalWrite(pinoLEDvermelho_pedestre, HIGH);
```

```
      delay(500);
      digitalWrite(pinoLEDvermelho_pedestre, LOW);
      delay(500);
    }

    // apaga LED vermelho
    digitalWrite(pinoLEDvermelho, LOW);
  }
}
```

Salve o sketch com o nome **atividade06**. O resultado esperado: ao pressionar o botão (pedestre), o sketch aguarda 1 minuto, apaga LED verde, acende LED amarelo, aguarda 15 segundos, apaga LED amarelo, acende LED vermelho e ao atingir esta condição, o LED vermelho do pedestre apaga e acende LED verde do pedestre. O sketch aguarda 30 segundos, apaga LED verde do pedestre e o LED vermelho do pedestre começa a piscar.

4.6 Atividade 7 – Medindo Temperaturas com NTC

Nesta atividade você aprenderá a reconhecer e usar o seguinte componente eletrônico:

- o NTC.

E o já conhecido:

- resistor.

Para encerrarmos este tutorial, propomos uma atividade bem simples em sua montagem, porém com conceitos que, juntamente com as atividades anteriores, darão a certeza de que o Arduino é uma plataforma simples, disponível para pessoas sem contato no dia-a-dia com a eletrônica e computação e capazes de realizar projetos no mundo real.

4.6.1 NTC

O NTC (Negative Temperature Coefficient – Coeficiente Negativo de Temperatura) é um componente eletrônico (termistor) sensível a temperatura. Conforme a **temperatura aumenta**, a sua **resistência diminui**.

Figura 4-27 – o NTC

Para um valor mais preciso da temperatura, aplica-se a equação Steinhart-Hart. Para quem gosta de cálculos, visite:

- http://en.wikipedia.org/wiki/Steinhart%E2%80%93Hart_equation

Nesta atividade simplificamos esta equação, que está disponível na íntegra em:

- http://www.arduino.cc/playground/ComponentLib/Thermistor2

O diagrama abaixo é a representação esquemática do circuito.

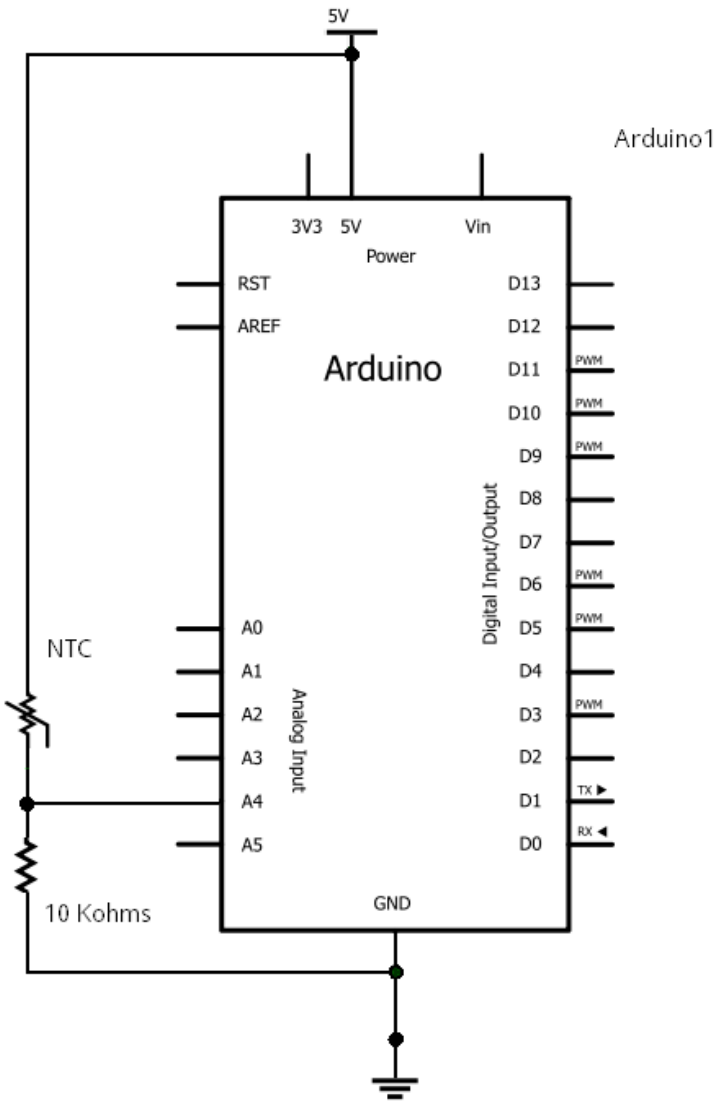

Figura 4-28 – diagrama atividade 07

Separe os seguintes componentes:

- um resistor de 10K ohms
- um NTC de 10K ohms

A figura seguinte é a placa Arduino conectada a *protoboard* com os respectivos componentes. Tente associar o esquema anterior com o desenho abaixo.

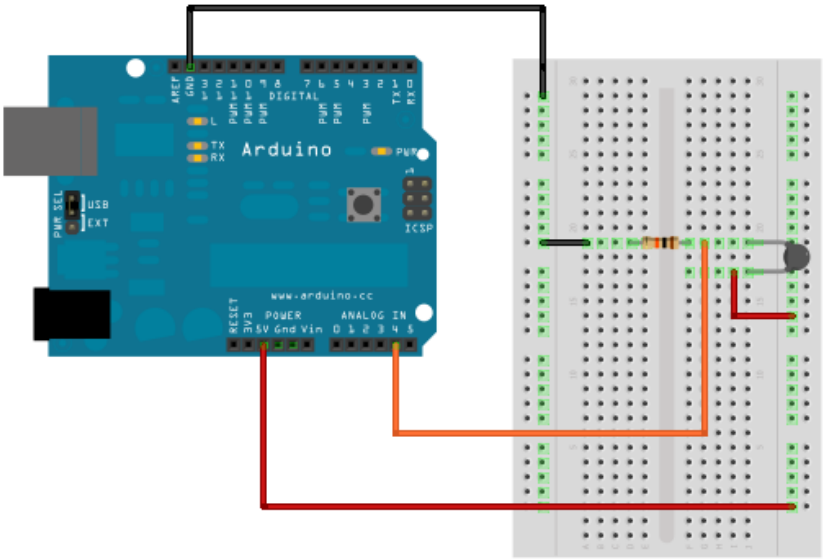

Figura 4-29 – interligando os componentes com a placa Arduino

Digite o sketch abaixo e transfira para a placa Arduino.

```
 /*
 atividade 07
 */

#include <math.h>

int pinoNTC   = 4;

int valorLido = 0;

double termistorNTC(int valorAnalogico) {
  double temperatura;
  temperatura = log(((10240000/valorAnalogico) - 10000));
  temperatura = 1 / (0.001129148 + (0.000234125 * temperatura) +
(0.0000000876741 * temperatura * temperatura * temperatura));
  //--- converte Kelvin em Celcius
  temperatura = temperatura - 273.15;
  return temperatura;
}

void setup() {
  pinMode(pinoNTC, INPUT);
  Serial.begin(9600);
  delay(5000);
}

void loop() {
  valorLido = int(termistorNTC(analogRead(pinoNTC)));
  Serial.println( valorLido );
  delay(1000);
}
```

Salve o sketch com o nome **atividade07**. O resultado esperado é ler a temperatura ambiente e mostrá-la no monitor serial da IDE do Arduino. Para isto, ao final da transferência do programa, é necessário clicar no botão **Serial Monitor** indicado na figura abaixo.

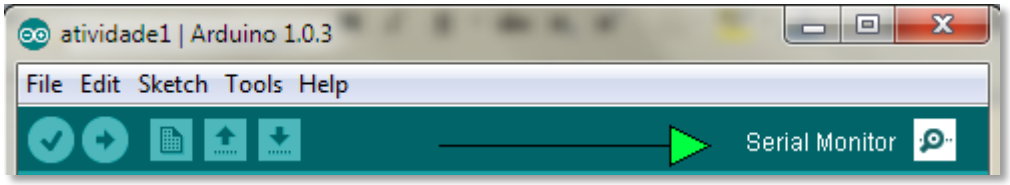

Figura 4-30 – botão monitor serial

Ao iniciar o sketch, o bloco setup() configura o pino que está ligado o NTC como entrada e define a velocidade de comunicação com a saída serial. Aguarda 5 segundos e transfere o controle para o bloco loop(). Este por sua vez chama a função **termistorNTC**, passando para esta função o valor informado pelo NTC. A função faz o cálculo, convertendo em graus Celcius, e devolve o valor calculado para a variável **valorLido**. Por fim, apresenta na tela do monitor serial este valor, aguarda 1 segundo e reinicia o ciclo.

Agora vamos explicar as novidades nesta atividade.

4.6.2 #include

#include <nome do arquivo>

#include é uma diretiva da linguagem Arduino que instrui a IDE, no momento de verificar e/ou transferir o sketch para a placa, de incluir o arquivo indicado. Então, **`#include <math.h>`** inclui o arquivo **math.h** no nosso sketch. Nesta atividade usamos a função matemática **log()** e por não ser algo trivial, como calcular o logaritmo de um número por exemplo, muitas outras funções da linguagem são armazenadas neste tipo de arquivo.

4.6.3 double

O tipo **double**, assim como o float, permite manipular números em ponto flutuante (ponto decimal). Até o momento, na linguagem Arduino estes tipos são exatamente iguais. Use somente onde realmente números inteiros não se aplicam.

4.6.4 Functions

Este não é um tópico novo, mas ainda não havíamos comentado. As funções (*functions*) são blocos de código que ajudam a estruturar um programa. Em todos os sketches, até por ser obrigatório, criamos duas funções: setup() e loop().

Toda função possui esta estrutura:

<tipo de dado do retorno> <nome da função> (<parâmetros de entrada>) {

}

e o comando **return** quando houver algum dado de retorno.

Parece complicado, não?! Então,

void setup() {

}

indica que a função setup não retorna dado (**void -** vazio) e não possui parâmetros de entrada **()**.

Nesta atividade,

```
double termistorNTC(int valorAnalogico) {
   .
   .
   .
  return temperatura;
}
```

indica que a função **termistorNTC** vai retornar um dado do tipo **double**, e vai receber como parâmetro um dado do tipo **int**. O retorno do dado do tipo double é feito pelo comando **return**.

4.6.5 log()

log(número)

Esta função retorna o logaritmo natural do número informado.

Obs. Estranhamente até a versão 0022, esta função não estava documentada no site oficial do Arduino.

4.6.6 Serial

Este objeto é usado para comunicação serial entre a placa Arduino e o computador ou até mesmo outro dispositivo que aceite comunicação serial. Quando em uso, o Arduino utiliza os pinos digitais 0 e 1 para a comunicação, portanto não use estes pinos para entrada ou saída.

Aliás, você deve ter notado que quando transferimos o sketch para a placa, dois LEDs associados a estes pinos piscam. Se não notou, transfira novamente o sketch e repare os LEDs piscarem.

Bom, voltando ao sketch...

No bloco setup(), `Serial.begin(9600);` inicia a comunicação com os pinos seriais a uma velocidade de 9600 bps (bits por segundo). Esta é a velocidade que a placa Arduino irá enviar os dados para o computador. Os valores comuns são: **300, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600,** ou **115200**. Porém, pode-se colocar outro valor caso o dispositivo com o qual você quer comunicar exija uma configuração diferente.

Já no bloco loop(), `Serial.println( valorLido );` exibe no monitor serial o conteúdo da variável **valorLido**.

5 Conclusão

Esperamos que você, no mínimo, tenha se divertido. Vimos uma pequena parte da plataforma Arduino e há muito ainda o que explorar. Para continuidade dos estudos, indicamos o curso "Linguagem Arduino I" e o tutorial sobre o Fritzing, mencionado anteriormente nesta apostila. Não deixe acompanhar as novidades m http://www.eaduino.com.br.

Muito obrigado.

Equipe EADuino.